```
/* ==== ecdet/__init__.py ==== */
"""ecdet —— 嵌入式软件代码缺陷与安全漏洞检测系统。
按技术方案 V1.0 的五层架构组织：
  L1 编译环境复现层（ecdet.l1_build）      —— 解决 P1：产出 compile_commands.json
  L2 统一 IR / 程序表示层（ecdet.l2_ir）   —— 统一告警数据模型与 SARIF 交换格式
  L3 多引擎分析层（ecdet.l3_analysis）     —— 规则引擎 / 外部引擎适配，硬件语义注入点
  L4 LLM 增强层（ecdet.l4_llm）            —— 告警分诊 / 误报过滤 / 修复建议（阶段三）
  L5 结果聚合与可证明性输出层（ecdet.l5_report） —— 去重、CWE/MISRA 映射、合规报告
"""
__version__ = "1.0.0"
/* ==== ecdet/cli.py ==== */
"""ecdet 命令行入口 —— 五层流水线的调度外壳。
命令：
  ecdet scan <path>        全流程：L1 构建复现 → L3 多引擎扫描 → L5 聚合输出
  ecdet init-ccdb <path>   仅执行 L1 构建复现（产出 compile_commands.json）
  ecdet rules              列出内置规则集
  ecdet report <sarif>     从 SARIF 结果重新生成 HTML 报告
"""
from __future__ import annotations
import argparse
import json
import sys
from pathlib import Path
from ecdet import __version__
from ecdet.l1_build import reconstruct
from ecdet.l2_ir import save_sarif
from ecdet.l3_analysis.engine import (
    DEFAULT_ENGINES,
    AnalysisTarget,
    all_rule_metas,
    resolve_enabled,
    run_all_engines,
)
from ecdet.l5_report import aggregate, build_compliance_records, render_html_report
_SKIP_DIRS = {".git", ".svn", "build", "ecdet_build", "node_modules", "__pycache__",
              ".venv", "tests"}
def _collect_sources(root: Path) -> list[Path]:
    out = []
    for p in sorted(root.rglob("*")):
        if p.suffix.lower() in (".c", ".h") and \
                not any(s in p.parts for s in _SKIP_DIRS):
            out.append(p.relative_to(root))
    return out
def cmd_scan(args: argparse.Namespace) -> int:
    root = Path(args.path).resolve()
    if not root.exists():
        print(f"错误：路径不存在 {root}", file=sys.stderr)
        return 2
    # L1 构建复现（--no-build 跳过，直接源码扫描）
    method = "skipped"
```

```
if not args.no_build:
    res = reconstruct(root)
    method = res.method
    for w in res.warnings:
        print(f"[L1] 警告：{w}")
# L3 多引擎扫描（增量模式：影响集收缩扫描范围，方案 3.3.6）
from ecdet.l3_analysis.engine import collect_sources
files = collect_sources(root)
if not files:
    print("错误：未发现 C 源文件", file=sys.stderr)
    return 2
old_fps = set()
out_dir = Path(args.output) if args.output else root / "ecdet_reports"
if args.baseline:
    from ecdet.incremental import load_baseline_fingerprints
    bp = Path(args.baseline)
    if bp.exists():
        old_fps = load_baseline_fingerprints(bp)
        print(f"[增量] 载入基线：{bp}（{len(old_fps)} 条指纹）")
    else:
        print(f"[增量] 基线文件不存在，按全量处理：{bp}")
if args.changed:
    from ecdet.incremental import compute_impact_set
    changed = [s.strip() for s in args.changed.split(",") if s.strip()]
    impact = compute_impact_set(root, changed)
    known = set(files)
    files = [f for f in files if f.as_posix() in set(impact.affected)]
    print(f"[增量] 影响集：{len(impact.affected)} 个文件"
          f"（改动 {len(changed)}，扫描收缩至 {len(files)}）")
    for f in files:
        print(f"    {f.as_posix()}  ← {impact.reason.get(f.as_posix(), '?')}")
    if not files:
        print("[增量] 影响集为空，无文件需要扫描")
        return 0
    _ = known
svd_path = None
if args.svd:
    svd_path = Path(args.svd)
else:
    candidates = [p for p in sorted(root.rglob("*.svd"))
                  if "ecdet_build" not in p.parts]
    svd_path = candidates[0] if candidates else None
if svd_path and not svd_path.is_absolute():
    svd_path = root / svd_path
target = AnalysisTarget(root=root, files=files, svd_path=svd_path)
if svd_path:
    print(f"[L2] 硬件语义：加载 SVD {svd_path.name}（MMIO 权限检查启用）")
alerts, metas = run_all_engines(target, profile=args.profile,
                                disable=args.disable, enable=args.enable)
# L5 聚合输出
```

```
    result = aggregate(alerts, metas)
    records = build_compliance_records(result)
    out_dir.mkdir(parents=True, exist_ok=True)
    sarif_path = save_sarif(alerts, metas, out_dir / "results.sarif")
    html_path = render_html_report(result, records,
                                   project_name=root.name,
                                   out_path=out_dir / "report.html")
    # 告警基线：差分 + 更新（方案 3.3.6）
    if old_fps or args.save_baseline:
        from ecdet.incremental import diff_baseline, save_baseline
        diff = diff_baseline(old_fps, alerts)
        base_path = save_baseline(alerts, out_dir / "baseline.json")
        if old_fps:
            print(f"[增量] 基线差分：{diff.summary}")
            for a in diff.new[:10]:
                print(f"    [新增] {a.rule_id} @ {a.location.path}:{a.location.line}")
            if len(diff.new) > 10:
                print(f"    … 其余 {len(diff.new) - 10} 条新增略")
        print(f"[增量] 基线已更新：{base_path}")
    s = result.stats
    print(f"\n=== ecdet 扫描完成 ===")
    print(f"工程：{root}")
    print(f"构建复现通道：{method} | 分析文件：{len(files)} 个")
    print(f"告警（去重后）：{s['total']} 条"
          f"（error {s['error']} / warning {s['warning']} / note {s['note']}）")
    if s.get("by_engine"):
        print(f"引擎贡献：{s['by_engine']}")
    if s.get("by_cwe"):
        top = sorted(s['by_cwe'].items(), key=lambda x: -x[1])[:5]
        print(f"CWE Top5：{top}")
    print(f"\nSARIF：{sarif_path}")
    print(f"HTML 报告：{html_path}")
    return 0 if s["error"] == 0 else 1
def cmd_init_ccdb(args: argparse.Namespace) -> int:
    root = Path(args.path).resolve()
    res = reconstruct(root)
    print(f"构建复现通道：{res.method}")
    print(f"编译命令：{len(res.commands)} 条")
    if res.target_hint:
        print(f"目标芯片：{res.target_hint}")
    for w in res.warnings:
        print(f"警告：{w}")
    if res.ccdb_path:
        print(f"编译数据库：{res.ccdb_path}")
        return 0
    return 1
def cmd_rules(args: argparse.Namespace) -> int:
    metas = all_rule_metas(DEFAULT_ENGINES)
    by_std: dict[str, list] = {}
    for m in metas:
```

```
        by_std.setdefault(m.standard, []).append(m)
    for std, ms in sorted(by_std.items()):
        print(f"\n[{std}]（{len(ms)} 条）")
        for m in sorted(ms, key=lambda x: x.rule_id):
            flag = "" if m.enabled_by_default else "（默认禁用）"
            print(f"  {m.rule_id:<28} {m.category:<10} {m.decidability:<10} {m.title_zh}{flag}")
    return 0
def cmd_report(args: argparse.Namespace) -> int:
    from ecdet.l2_ir import load_sarif
    alerts, metas = load_sarif(args.sarif)
    result = aggregate(alerts, metas)
    records = build_compliance_records(result)
    out = Path(args.output) if args.output else Path(args.sarif).with_suffix(".html")
    path = render_html_report(result, records, out_path=out)
    print(f"HTML 报告：{path}")
    return 0
def cmd_triage(args: argparse.Namespace) -> int:
    """L4 告警分诊：SARIF + 工程上下文 → 分诊结论回写 SARIF/HTML 报告。"""
    from ecdet.l2_ir import load_sarif
    from ecdet.l4_llm.client import MockLlmClient, OpenAICompatClient
    from ecdet.l4_llm.triage import triage_alerts
    sarif_path = Path(args.sarif)
    if not sarif_path.exists():
        print(f"错误：SARIF 不存在 {sarif_path}", file=sys.stderr)
        return 2
    project = Path(args.project).resolve() if args.project else sarif_path.parent.parent
    alerts, metas = load_sarif(sarif_path)
    if not alerts:
        print("SARIF 中无告警，无需分诊")
        return 0
    # 增量研判：仅对基线之外的新增告警调用 LLM（方案 3.3.6）
    if args.only_new:
        from ecdet.incremental import load_baseline_fingerprints
        old_fps = load_baseline_fingerprints(args.only_new)
        n_skip = sum(1 for a in alerts if a.ensure_fingerprint() in old_fps)
        alerts = [a for a in alerts if a.fingerprint not in old_fps]
        print(f"[增量研判] 基线持续告警 {n_skip} 条跳过，"
              f"仅研判新增 {len(alerts)} 条")
        if not alerts:
            print("无新增告警，完成")
            return 0
    if args.mock:
        client = MockLlmClient()
        print("[L4] 使用 Mock LLM（离线演示模式）")
    else:
        client = OpenAICompatClient(base_url=args.llm_url, model=args.model)
        print(f"[L4] LLM 端点：{client.base_url}（模型 {client.model}）")
    verdicts, calls = triage_alerts(alerts, project, client=client,
                                    budget_chars=args.budget)
    by_v: dict[str, int] = {}
```

```
    for v in verdicts:
        by_v[v.verdict] = by_v.get(v.verdict, 0) + 1
    result = aggregate(alerts, metas)
    records = build_compliance_records(result)
    out_dir = Path(args.output) if args.output else sarif_path.parent
    out_dir.mkdir(parents=True, exist_ok=True)
    save_sarif(alerts, metas, out_dir / "results.sarif")
    render_html_report(result, records, project_name=project.name,
                       out_path=out_dir / "report.html")
    print("\n=== L4 分诊完成 ===")
    print(f"告警：{len(verdicts)} 条 | LLM 调用：{calls} 次")
    for k in ("TRUE_POSITIVE", "FALSE_POSITIVE", "NEEDS_REVIEW"):
        if by_v.get(k):
            print(f"  {k}: {by_v[k]}")
    print(f"\n分诊后 SARIF：{out_dir / 'results.sarif'}")
    print(f"HTML 报告：{out_dir / 'report.html'}")
    return 0
def cmd_fix(args: argparse.Namespace) -> int:
    """L4 修复建议 + ToolLoopVerify 验证闭环。"""
    from ecdet.l2_ir import load_sarif
    from ecdet.l3_analysis.builtin_rules.checks import build_file_facts, finalize_facts
    from ecdet.l4_llm.client import MockLlmClient, OpenAICompatClient
    from ecdet.l4_llm.fixer import FixProposer, FixVerifier, MockFixClient
    sarif_path = Path(args.sarif)
    if not sarif_path.exists():
        print(f"错误：SARIF 不存在 {sarif_path}", file=sys.stderr)
        return 2
    project = Path(args.project).resolve() if args.project else sarif_path.parent.parent
    alerts, metas = load_sarif(sarif_path)
    # 只对真缺陷生成补丁（已分诊时取 TRUE_POSITIVE；未分诊时取全部）
    targets = [a for a in alerts
               if (a.triage or {}).get("verdict", "TRUE_POSITIVE") == "TRUE_POSITIVE"]
    if args.max:
        targets = targets[:args.max]
    if not targets:
        print("无待修复告警（无 TRUE_POSITIVE 结论）")
        return 0
    # 构建工程事实
    need = sorted({a.location.path.replace("\\", "/") for a in targets})
    sources: dict[str, str] = {}
    facts = []
    for rel in need:
        p = project / rel
        if p.exists():
            src = p.read_text(encoding="utf-8", errors="replace")
            sources[rel] = src
            facts.append(build_file_facts(rel, src))
    ctx = finalize_facts(facts)
    client = MockFixClient() if args.mock else OpenAICompatClient(
        base_url=args.llm_url, model=args.model)
```

```
    mode = "Mock（离线演示）" if args.mock else client.base_url
    print(f"[L4] 修复建议 LLM: {mode}")
    proposer = FixProposer(client)
    verifier = FixVerifier(project, use_compiler=not args.no_compile)
    out_dir = (Path(args.output) if args.output else sarif_path.parent) / "fixes"
    out_dir.mkdir(parents=True, exist_ok=True)
    accepted, rejected = 0, 0
    records = []
    for i, a in enumerate(targets, 1):
        proposal = proposer.propose(a, ctx)
        if not proposal.fail_reason:
            src = sources.get(a.location.path.replace("\\", "/"))
            if src is not None:
                proposal = verifier.verify(proposal, ctx, src)
        tag = "采纳" if proposal.verified else "拒绝"
        accepted += int(proposal.verified)
        rejected += int(not proposal.verified)
        print(f"  [{tag}] {a.rule_id} @ {a.location.path}:{a.location.line}"
              + ("" if proposal.verified else f" —— {proposal.fail_reason}"))
        records.append({
            "rule_id": a.rule_id, "path": a.location.path,
            "line": a.location.line, "func": proposal.func_name,
            "verified": proposal.verified,
            "fail_reason": proposal.fail_reason,
            "explanation": proposal.explanation, "diff": proposal.diff,
        })
        if proposal.verified and proposal.diff:
            patch_name = f"{i:02d}_{a.rule_id.replace(' ', '_').replace(':', '')}_" \
                         f"{Path(a.location.path).stem}.patch"
            (out_dir / patch_name).write_text(proposal.diff + "\n",
                                              encoding="utf-8")
    (out_dir / "fixes.json").write_text(
        json.dumps({"accepted": accepted, "rejected": rejected,
                    "proposals": records}, ensure_ascii=False, indent=2),
        encoding="utf-8")
    print(f"\n=== L4 修复闭环完成 ===")
    print(f"建议 {len(records)} 条：采纳 {accepted} / 拒绝 {rejected}"
          f"（验证：静态重扫 + {'gcc 语法检查' if not args.no_compile else '未启用编译检查'}）")
    print(f"补丁与记录：{out_dir}")
    return 0
def cmd_impact(args: argparse.Namespace) -> int:
    """影响集计算：changed → include 反向闭包 → 调用图反向闭包。"""
    from ecdet.incremental import compute_impact_set
    root = Path(args.path).resolve()
    if not root.exists():
        print(f"错误：路径不存在 {root}", file=sys.stderr)
        return 2
    changed = [s.strip() for s in args.changed.split(",") if s.strip()]
    impact = compute_impact_set(root, changed)
    if args.json:
```

```
        print(json.dumps(impact.as_json(), ensure_ascii=False, indent=2))
        return 0
    print(f"变更 {len(impact.changed)} 个文件 → 影响集 {len(impact.affected)} 个文件：")
    for f in impact.affected:
        print(f"  {f}  ← {impact.reason.get(f, '?')}")
    return 0
def cmd_orchestrate(args: argparse.Namespace) -> int:
    """六智能体协同全流程（方案 3.3.7 Orchestrate）。"""
    from ecdet.l4_llm.agents import Orchestrator, OrchestratorConfig
    from ecdet.l5_report import aggregate, build_compliance_records, render_html_report
    root = Path(args.path).resolve()
    if not root.exists():
        print(f"错误：路径不存在 {root}", file=sys.stderr)
        return 2
    cfg = OrchestratorConfig(
        mock=args.mock, llm_url=args.llm_url, llm_model=args.model,
        use_llm=not args.no_llm, use_dynamic=not args.no_dynamic,
        single_agent=args.single_agent, auto_accept=not args.no_auto_accept,
        max_fix_targets=args.max_fix)
    mode = "单智能体串行（A6）" if args.single_agent else "多智能体协同"
    print(f"[L4] 编排模式：{mode} | LLM：{'Mock' if args.mock else 'OpenAI 兼容端点'}"
          f" | 动态通道：{'开' if cfg.use_dynamic else '关'}")
    state = Orchestrator(cfg).orchestrate(root)
    for line in state.logs:
        print(f"  {line}")
    # L5 聚合与报告
    result = aggregate(state.static_alerts, state.rule_metas)
    records = build_compliance_records(result)
    out_dir = root / "ecdet_reports"
    out_dir.mkdir(parents=True, exist_ok=True)
    save_sarif(state.static_alerts, state.rule_metas, out_dir / "results.sarif")
    html = render_html_report(result, records, project_name=root.name,
                              out_path=out_dir / "report.html")
    if state.review_queue:
        print("\n=== 人工复核闸门（高危误报判定） ===")
        for item in state.review_queue:
            print(f"  {item['rule_id']} @ {item['path']}:{item['line']}")
            print(f"    LLM 理由：{item['llm_reason'][:80]}")
    print(f"\n=== 编排完成 ===")
    print(f"告警 {len(state.static_alerts)} 条 | 动态触发 "
          f"{len(state.dynamic_findings)} | 监控升级 "
          f"{len(state.monitor_result.upgraded) if state.monitor_result else 0} | "
          f"修复采纳 {sum(1 for f in state.fixes if f.verified)}")
    print(f"LLM 调用 {state.llm_calls} 次 | 耗时 {state.seconds:.1f}s")
    print(f"HTML 报告：{html}")
    return 0
def cmd_serve(args: argparse.Namespace) -> int:
    """Web 控制台：浏览器注册工程 → 后台扫描 → 告警浏览/复核。"""
    from ecdet.webui import WebUI
    data_dir = Path(args.data_dir).resolve()
```

```
    print(f"数据目录：{data_dir}")
    WebUI(data_dir, port=args.port).serve()
    return 0
def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(
        prog="ecdet",
        description="嵌入式软件代码缺陷与安全漏洞检测系统",
    )
    parser.add_argument("--version", action="version", version=f"ecdet {__version__}")
    sub = parser.add_subparsers(dest="command", required=True)
    p_scan = sub.add_parser("scan", help="全流程扫描")
    p_scan.add_argument("path", help="工程根目录")
    p_scan.add_argument("--profile", choices=["automotive", "industrial"],
                        help="行业配置文件（规则裁剪）")
    p_scan.add_argument("--disable", nargs="*", default=[], help="禁用规则 ID")
    p_scan.add_argument("--enable", nargs="*", default=[], help="额外启用规则 ID")
    p_scan.add_argument("--no-build", action="store_true",
                        help="跳过 L1 构建复现，直接源码扫描")
    p_scan.add_argument("--svd", help="CMSIS-SVD 芯片描述文件（缺省自动发现工程内 *.svd）")
    p_scan.add_argument("--baseline", help="上次扫描的基线 JSON（存在则差分并仅报新增）")
    p_scan.add_argument("--save-baseline", action="store_true",
                        help="扫描后将告警指纹存为基线（输出目录 baseline.json）")
    p_scan.add_argument("--changed", help="增量模式：逗号分隔的变更文件（相对根目录），"
                                          "按影响集收缩扫描范围")
    p_scan.add_argument("--output", "-o", help="报告输出目录")
    p_scan.set_defaults(func=cmd_scan)
    p_ccdb = sub.add_parser("init-ccdb", help="L1 构建复现")
    p_ccdb.add_argument("path", help="工程根目录")
    p_ccdb.set_defaults(func=cmd_init_ccdb)
    p_rules = sub.add_parser("rules", help="列出规则集")
    p_rules.set_defaults(func=cmd_rules)
    p_report = sub.add_parser("report", help="从 SARIF 生成 HTML 报告")
    p_report.add_argument("sarif", help="SARIF 文件路径")
    p_report.add_argument("--output", "-o", help="输出 HTML 路径")
    p_report.set_defaults(func=cmd_report)
    p_triage = sub.add_parser("triage", help="L4 LLM 告警分诊（误报过滤/证据核对）")
    p_triage.add_argument("sarif", help="scan 产出的 results.sarif 路径")
    p_triage.add_argument("--project", help="工程根目录（告警相对路径的基准，"
                                            "缺省取 SARIF 上两级）")
    p_triage.add_argument("--llm-url", help="OpenAI 兼容端点"
                                            "（缺省 env ECODET_LLM_BASE_URL 或 vLLM 默认端口）")
    p_triage.add_argument("--model", help="模型名（缺省 env ECODET_LLM_MODEL）")
    p_triage.add_argument("--mock", action="store_true",
                          help="使用确定性 Mock LLM（离线演示/测试）")
    p_triage.add_argument("--budget", type=int, default=8000,
                          help="上下文字符预算（默认 8000）")
    p_triage.add_argument("--only-new", help="增量研判：仅研判基线 JSON 之外的新增告警")
    p_triage.add_argument("--output", "-o", help="输出目录（缺省覆盖原 SARIF 所在目录）")
    p_triage.set_defaults(func=cmd_triage)
    p_fix = sub.add_parser("fix", help="L4 修复建议生成 + ToolLoopVerify 验证闭环")
```

```
    p_fix.add_argument("sarif", help="results.sarif 路径（建议先 triage）")
    p_fix.add_argument("--project", help="工程根目录（缺省取 SARIF 上两级）")
    p_fix.add_argument("--llm-url", help="OpenAI 兼容端点（缺省 env）")
    p_fix.add_argument("--model", help="模型名（缺省 env）")
    p_fix.add_argument("--mock", action="store_true", help="使用确定性 Mock LLM")
    p_fix.add_argument("--no-compile", action="store_true",
                       help="跳过 gcc 语法检查（仅静态重扫验证）")
    p_fix.add_argument("--max", type=int, default=20, help="单次处理告警数上限")
    p_fix.add_argument("--output", "-o", help="输出目录")
    p_fix.set_defaults(func=cmd_fix)
    p_impact = sub.add_parser("impact", help="增量影响集计算（方案 3.3.6）")
    p_impact.add_argument("path", help="工程根目录")
    p_impact.add_argument("--changed", required=True,
                          help="逗号分隔的变更文件（相对根目录）")
    p_impact.add_argument("--json", action="store_true", help="输出 JSON")
    p_impact.set_defaults(func=cmd_impact)
    p_orch = sub.add_parser("orchestrate", help="六智能体协同全流程（3.3.7）")
    p_orch.add_argument("path", help="工程根目录")
    p_orch.add_argument("--svd", help="CMSIS-SVD 文件（缺省自动发现）")
    p_orch.add_argument("--mock", action="store_true", help="Mock LLM（离线演示）")
    p_orch.add_argument("--llm-url", help="OpenAI 兼容端点（缺省 env）")
    p_orch.add_argument("--model", help="模型名（缺省 env）")
    p_orch.add_argument("--no-llm", action="store_true", help="跳过 L4 研判/修复（A1）")
    p_orch.add_argument("--no-dynamic", action="store_true",
                        help="跳过动态通道与运行时监控（A5）")
    p_orch.add_argument("--single-agent", action="store_true",
                        help="单智能体串行模式（A6 消融）")
    p_orch.add_argument("--no-auto-accept", action="store_true",
                        help="高危误报判定挂起待人工（缺省自动接受并留痕）")
    p_orch.add_argument("--max-fix", type=int, default=5, help="单次修复目标上限")
    p_orch.set_defaults(func=cmd_orchestrate)
    p_serve = sub.add_parser("serve", help="启动 Web 控制台（零依赖）")
    p_serve.add_argument("--port", type=int, default=8600)
    p_serve.add_argument("--data-dir", default="webui_data",
                         help="数据目录（SQLite + 工作区 + 运行产物）")
    p_serve.set_defaults(func=cmd_serve)
    args = parser.parse_args(argv)
    return args.func(args)
if __name__ == "__main__":
    sys.exit(main())
/* ==== ecdet/eval.py ==== */
"""对比评测与消融实验框架（方案 3.5，对应 P4/P5/P6 验证）。
指标（方案 3.5.2）：
  - 函数级 Precision/Recall/F1（主口径）与告警级计数；
  - 每千行分析耗时（全量模式）；
  - 可证明性比例（Proven 级结论占比）；
  - LLM 误报过滤量（分诊 FALSE_POSITIVE 数，体现 L4 净贡献）。
消融配置（方案 3.5.3 表 3-3）：
  A0 完整系统 / A1 去 LLM / A2 去硬件语义 / A3 去 RAG。
  A4（去验证闭环）作用于修复建议质量，见 ecdet.l4_llm.fixer 的验收率对比。
```

```
"""
from __future__ import annotations
import json
import time
from dataclasses import dataclass, field
from pathlib import Path
from ecdet.l2_ir.model import Alert
from ecdet.l3_analysis.builtin_rules.rules_meta import BUILTIN_RULES
from ecdet.l3_analysis.engine import AnalysisTarget, BuiltinEngine
from ecdet.l4_llm.client import MockLlmClient
from ecdet.l4_llm.triage import triage_alerts
HW_RULE_IDS = {"ECDET-MMIO-ACCESS", "ECDET-ISR-RACE", "ECDET-VOLATILE",
               "ECDET-ISR-BLOCK"}
SKIP_DIRS = {".git", "build", "ecdet_build", "ecdet_reports", ".venv",
             "__pycache__", "node_modules"}
# ---------------------------------------------------------------------------
# 基准真值
# ---------------------------------------------------------------------------
@dataclass
class GTEntry:
    path: str
    func_name: str
    start_line: int
    end_line: int
    label: str                    # vulnerable / clean
    cwe: str = ""
def load_ground_truth(path: str | Path) -> list[GTEntry]:
    out = []
    for line in Path(path).read_text(encoding="utf-8").splitlines():
        line = line.strip()
        if not line:
            continue
        d = json.loads(line)
        out.append(GTEntry(
            path=d["path"].replace("\\", "/"),
            func_name=d.get("func_name", ""),
            start_line=int(d["start_line"]),
            end_line=int(d["end_line"]),
            label=d.get("label", "clean"),
            cwe=d.get("cwe", ""),
        ))
    return out
def gt_from_corpus(corpus: Path, vulnerable_names: set[str],
                   exclude: set[str] | None = None) -> list[GTEntry]:
    """由语料自身切片生成基准真值（评测管线的自洽测试入口）。
    生产评测应以人工标注的 gt.jsonl 为准（方案 3.5.1）。
    """
    from ecdet.l3_analysis.builtin_rules.checks import build_file_facts
    exclude = exclude or set()
    out = []
```

```
    for p in sorted(corpus.rglob("*")):
        if p.suffix.lower() != ".c" or any(s in p.parts for s in SKIP_DIRS):
            continue
        rel = p.relative_to(corpus).as_posix()
        facts = build_file_facts(rel, p.read_text(encoding="utf-8",
                                                  errors="replace"))
        for func in facts.functions:
            if func.name in exclude:
                continue
            out.append(GTEntry(
                path=rel, func_name=func.name, start_line=func.def_line,
                end_line=func.body_close_line,
                label="vulnerable" if func.name in vulnerable_names else "clean"))
    return out
# ---------------------------------------------------------------------------
# 消融配置与执行
# ---------------------------------------------------------------------------
@dataclass
class AblationConfig:
    name: str
    use_llm: bool = True
    use_hw: bool = True
    use_rag: bool = True
    use_dynamic: bool = True       # A5 关（去动态通道）
    single_agent: bool = False     # A6 开（单智能体串行）
    profile: str = ""              # 规则配置文件（如 juliet：屏蔽工程级规则）
    note: str = ""
DEFAULT_ABLATIONS = [
    AblationConfig("A0 完整系统", note="双通道+硬件语义+LLM+RAG"),
    AblationConfig("A1 去 LLM", use_llm=False, note="仅工具流水线（对应 P5）"),
    AblationConfig("A2 去硬件语义", use_hw=False, note="移除 ISR/MMIO 模型（对应 P2）"),
    AblationConfig("A3 去 RAG", use_rag=False, note="研判不接知识检索"),
    AblationConfig("A5 去动态通道", use_dynamic=False,
                   note="仅静态通道（对应 P7 双通道互补）"),
    AblationConfig("A6 单智能体串行", single_agent=True,
                   note="多智能体退化为顺序处理（对照 Agentless）"),
]
@dataclass
class ToolRun:
    name: str
    note: str = ""
    alerts: list[Alert] = field(default_factory=list)
    seconds: float = 0.0
    kloc: float = 0.0
    provenance_ratio: float = 0.0
    triage_counts: dict = field(default_factory=dict)
    available: bool = True
def _corpus_files(corpus: Path) -> list[Path]:
    return [p.relative_to(corpus) for p in sorted(corpus.rglob("*"))
            if p.suffix.lower() in (".c", ".h")
```

```
            and not any(s in p.parts for s in SKIP_DIRS)]
def _kloc(corpus: Path) -> float:
    total = 0
    for rel in _corpus_files(corpus):
        total += (corpus / rel).read_text(encoding="utf-8",
                                          errors="replace").count("\n") + 1
    return total / 1000.0
def run_ecdet_config(corpus: Path, config: AblationConfig,
                     svd_path: Path | None = None,
                     llm_client=None,
                     rag_conf_threshold: float = 0.6) -> ToolRun:
    """跑一个消融配置：扫描 →（可选）分诊过滤误报 → 计时与统计。"""
    files = _corpus_files(corpus)
    enabled = {r.rule_id for r in BUILTIN_RULES}
    if not config.use_hw:
        enabled -= HW_RULE_IDS
    if config.profile:
        from ecdet.l3_analysis.engine import INDUSTRY_PROFILES
        enabled -= INDUSTRY_PROFILES.get(config.profile, set())
    svd = svd_path if config.use_hw else None
    t0 = time.perf_counter()
    target = AnalysisTarget(root=corpus, files=files, svd_path=svd)
    alerts = BuiltinEngine().run(target, enabled)
    scan_seconds = time.perf_counter() - t0
    triage_counts: dict = {}
    if config.use_llm and alerts:
        client = llm_client or MockLlmClient()
        t1 = time.perf_counter()
        _, _ = triage_alerts(alerts, corpus, client=client,
                             rag_conf_threshold=(rag_conf_threshold
                                                 if config.use_rag else 0.0))
        scan_seconds += time.perf_counter() - t1
        # A6 消融的组织学差异：多智能体并行扇出 vs 单智能体顺序处理
        if not config.single_agent and len(alerts) > 4:
            from concurrent.futures import ThreadPoolExecutor
            with ThreadPoolExecutor(max_workers=4) as pool:
                list(pool.map(lambda a: a.ensure_fingerprint(), alerts))
        for a in alerts:
            v = (a.triage or {}).get("verdict", "UNTRIAGED")
            triage_counts[v] = triage_counts.get(v, 0) + 1
        alerts = [a for a in alerts
                  if (a.triage or {}).get("verdict") != "FALSE_POSITIVE"]
    # L3 动态通道（3.3.5）：真触发 PoC → Detected 级告警
    if config.use_dynamic:
        from ecdet.l3_analysis.dynamic import DynamicEngine
        from ecdet.l3_analysis.builtin_rules.checks import build_file_facts
        engine = DynamicEngine(max_functions=8, max_seeds=17)
        if engine.available:
            t2 = time.perf_counter()
            facts = [build_file_facts(rel.as_posix(),
```

```
                                    (corpus / rel).read_text(
                                        encoding="utf-8", errors="replace"))
                        for rel in files if rel.as_posix().endswith(".c")]
            findings = engine.run(corpus, facts)
            scan_seconds += time.perf_counter() - t2
            alerts = alerts + [f_.to_alert() for f_ in findings]
    n_prov = sum(1 for a in alerts if a.provenance.value == "proven")
    return ToolRun(
        name=config.name, note=config.note, alerts=alerts,
        seconds=scan_seconds, kloc=_kloc(corpus),
        provenance_ratio=(n_prov / len(alerts)) if alerts else 0.0,
        triage_counts=triage_counts,
    )
def run_baseline(corpus: Path, engine) -> ToolRun:
    """外部基线（cppcheck/clang-tidy 等适配器实例）。不可用时 available=False。"""
    name = engine.name
    if not engine.available():
        return ToolRun(name=f"{name}（基线）", note="未安装，跳过", available=False)
    files = _corpus_files(corpus)
    t0 = time.perf_counter()
    alerts = engine.run(AnalysisTarget(root=corpus, files=files), set())
    seconds = time.perf_counter() - t0
    return ToolRun(name=f"{name}（基线）", alerts=alerts, seconds=seconds,
                   kloc=_kloc(corpus))
# ---------------------------------------------------------------------------
# 指标
# ---------------------------------------------------------------------------
def _in_func(alert: Alert, g: GTEntry) -> bool:
    return (alert.location.path.replace("\\", "/") == g.path
            and g.start_line <= alert.location.line <= g.end_line)
def compute_metrics(alerts: list[Alert], gt: list[GTEntry]) -> dict:
    """函数级 P/R/F1（主口径）+ 告警级计数（方案 3.5.2 双口径）。"""
    vuln = [g for g in gt if g.label == "vulnerable"]
    clean = [g for g in gt if g.label == "clean"]
    tp = sum(1 for g in vuln if any(_in_func(a, g) for a in alerts))
    fn = len(vuln) - tp
    fp = sum(1 for g in clean if any(_in_func(a, g) for a in alerts))
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    f1 = (2 * precision * recall / (precision + recall)
          if (precision + recall) else 0.0)
    tp_a = sum(1 for a in alerts if any(_in_func(a, g) for g in vuln))
    fp_a = sum(1 for a in alerts if any(_in_func(a, g) for g in clean))
    # 动态通道贡献率（方案 3.5.2 之 6）：动态新发现且静态未报 / 全部已确认
    static_hit = {g.func_name for g in vuln
                  if any(_in_func(a, g) and a.engine != "dynamic" for a in alerts)}
    dyn_hit = {g.func_name for g in vuln
               if any(_in_func(a, g) and a.engine == "dynamic" for a in alerts)}
    detected_total = len(static_hit | dyn_hit)
    dyn_only = len(dyn_hit - static_hit)
```

```
    return {
        "func_tp": tp, "func_fn": fn, "func_fp": fp,
        "vulnerable_total": len(vuln), "clean_total": len(clean),
        "precision": round(precision, 4),
        "recall": round(recall, 4),
        "f1": round(f1, 4),
        "alert_tp": tp_a, "alert_fp": fp_a,
        "alert_outside_gt": len(alerts) - tp_a - fp_a,
        "dynamic_contribution": (round(dyn_only / detected_total, 4)
                                 if detected_total else 0.0),
        "dynamic_only_funcs": sorted(dyn_hit - static_hit),
    }
# ---------------------------------------------------------------------------
# 报告
# ---------------------------------------------------------------------------
def render_markdown(runs: list[tuple[ToolRun, dict]]) -> str:
    """评测结果表（填充方案 V1.2 表 3-5 模板，含动态贡献率列）。"""
    lines = [
        "# ecdet 评测结果（方案 V1.2 表 3-5，七配置消融）",
        "",
        "| 系统配置 | 精确率 | 召回率 | F1 | 函数TP/FN/FP | 告警TP/FP | "
        "千行耗时(s) | 可证明性比例 | 动态贡献率 | LLM过滤 |",
        "|---|---|---|---|---|---|---|---|---|---|",
    ]
    for run, m in runs:
        if not run.available:
            lines.append(f"| {run.name} | - | - | - | - | - | - | - | - | "
                         f"{run.note} |")
            continue
        kloc_time = round(run.seconds / run.kloc, 3) if run.kloc else 0.0
        filtered = run.triage_counts.get("FALSE_POSITIVE", 0)
        has_dynamic = any(a.engine == "dynamic" for a in run.alerts)
        dyn_col = f"{m.get('dynamic_contribution', 0):.0%}" if has_dynamic else "-"
        lines.append(
            f"| {run.name} | {m['precision']:.2%} | {m['recall']:.2%} | "
            f"{m['f1']:.2%} | {m['func_tp']}/{m['func_fn']}/{m['func_fp']} | "
            f"{m['alert_tp']}/{m['alert_fp']} | {kloc_time} | "
            f"{run.provenance_ratio:.1%} | {dyn_col} | {filtered} |")
    lines.append("")
    lines.append("> 函数级口径：告警落入函数行区间即视为覆盖该函数；"
                 "clean 函数出现任一告警计一次 FP。"
                 "动态贡献率 = 动态新发现且静态未报的漏洞函数占全部已确认漏洞函数之比"
                 "（方案 3.5.2 之 6）。")
    return "\n".join(lines)
/* ==== ecdet/incremental.py ==== */
"""增量分析与告警基线管理（方案 3.3.6）。
影响集计算：修改文件 → 头文件包含闭包 → 调用图反向闭包 → 受影响文件集合；
告警基线：参照 CodeChecker 的存储/差分机制，新旧告警按指纹差分为
新增 / 已解决 / 持续三类，仅新增告警进入 L4 研判，使深度分析可以
嵌入每日集成而非一次性离线行为。
```

```
局限（MVP）：指纹含行号，告警因代码移动而漂移时计为"新增+已解决"
（CodeChecker 以上下文哈希缓解，留待阶段五）。
"""
from __future__ import annotations
import json
import re
from dataclasses import dataclass, field
from pathlib import Path
from ecdet.l2_ir.model import Alert
from ecdet.l3_analysis.builtin_rules.checks import build_file_facts, finalize_facts
# 匹配原始源码行中的本地包含（PPLine.text 的引号内容已被遮蔽，不可用）
INCLUDE_LOCAL_RE = re.compile(r'#\s*include\s*"([^"]+)"')
# ---------------------------------------------------------------------------
# 影响集
# ---------------------------------------------------------------------------
@dataclass
class ImpactSet:
    changed: list[str] = field(default_factory=list)
    affected: list[str] = field(default_factory=list)      # 全闭包（含 changed）
    reason: dict[str, str] = field(default_factory=dict)   # 文件 → 纳入原因
    def as_json(self) -> dict:
        return {"changed": self.changed, "affected": self.affected,
                "reason": self.reason}
def _norm(p: str) -> str:
    return p.replace("\\", "/")
def compute_impact_set(root: str | Path, changed: list[str]) -> ImpactSet:
    """方案 3.3.6 影响集：include 反向闭包 + 调用图反向闭包。
    changed 中的文件必须相对 root。返回受影响文件集合（相对路径，posix）。
    """
    root = Path(root)
    changed_norm = [_norm(c) for c in changed]
    changed_set = set(changed_norm)
    # 全工程事实（一次构建，供 include 图与调用图共用）
    paths: list[str] = []
    for p in sorted(root.rglob("*")):
        if p.suffix.lower() in (".c", ".h") and not any(
                s in p.parts for s in (".git", "build", "ecdet_build", ".venv")):
            paths.append(p.relative_to(root).as_posix())
    facts = [build_file_facts(rel, (root / rel).read_text(encoding="utf-8",
                                                          errors="replace"))
             for rel in paths]
    ctx = finalize_facts(facts)
    # 1) include 图：includer → {included}；解析 "local.h" 相对文件目录再相对 root
    # 注意：PPLine.text 是遮蔽后文本（引号内容已清空），必须读原始行
    include_edges: dict[str, set[str]] = {}
    for f in ctx.facts:
        edges = include_edges.setdefault(_norm(f.path), set())
        base_dir = _norm(str(Path(f.path).parent))
        for p in f.lex.pp_lines:
            if p.directive != "include":
```

```
            continue
        raw_line = (f.lex.source_lines[p.line - 1]
                    if p.line - 1 < len(f.lex.source_lines) else "")
        m = INCLUDE_LOCAL_RE.search(raw_line)
        if not m:
            continue
        target = _norm(m.group(1))
        for cand in (f"{base_dir}/{target}" if base_dir != "." else target,
                     target):
            if cand in set(paths):
                edges.add(cand)
                break
# 反向 include 闭包（多轮扩散）
affected = set(changed_set)
reason = {c: "changed" for c in changed_norm}
frontier = set(changed_set)
while frontier:
    nxt: set[str] = set()
    for includer, included in include_edges.items():
        if includer in affected:
            continue
        if included & frontier:
            affected.add(includer)
            reason[includer] = f"包含受影响头文件（{sorted(included & frontier)[0]}）"
            nxt.add(includer)
    frontier = nxt
# 2) 调用图反向闭包：文件 A 中函数调用受影响文件中定义的函数 → A 受影响
func_home: dict[str, str] = {}
for f in ctx.facts:
    for func in f.functions:
        func_home.setdefault(func.name, _norm(f.path))
file_calls: dict[str, set[str]] = {}
for f in ctx.facts:
    callees: set[str] = set()
    for func in f.functions:
        callees.update(name for name, _ in func.calls_in_body())
    file_calls[_norm(f.path)] = callees
frontier = {a for a in affected if a.endswith(".c") or a.endswith(".h")}
while frontier:
    nxt: set[str] = set()
    for caller, callees in file_calls.items():
        if caller in affected:
            continue
        hit_callees = {c for c in callees
                       if func_home.get(c) in frontier}
        if hit_callees:
            affected.add(caller)
            home = sorted({func_home[c] for c in hit_callees})[0]
            reason[caller] = f"调用受影响文件的函数（{home}）"
            nxt.add(caller)
```

```
        frontier = nxt
    return ImpactSet(changed=changed_norm, affected=sorted(affected),
                     reason=reason)
# ---------------------------------------------------------------------------
# 告警基线
# ---------------------------------------------------------------------------
def save_baseline(alerts: list[Alert], path: str | Path) -> Path:
    """保存当前告警指纹为基线（方案 3.3.6：CodeChecker 式存储）。"""
    out = Path(path)
    out.parent.mkdir(parents=True, exist_ok=True)
    payload = {
        "fingerprints": sorted({a.ensure_fingerprint() for a in alerts}),
        "count": len(alerts),
    }
    out.write_text(json.dumps(payload, ensure_ascii=False, indent=2),
                   encoding="utf-8")
    return out
def load_baseline_fingerprints(path: str | Path) -> set[str]:
    data = json.loads(Path(path).read_text(encoding="utf-8"))
    return set(data.get("fingerprints", []))
@dataclass
class BaselineDiff:
    """基线差分结果。已解决仅计数（基线只存指纹，不存告警详情）。"""
    new: list[Alert] = field(default_factory=list)
    persisting: list[Alert] = field(default_factory=list)
    resolved_count: int = 0
    @property
    def summary(self) -> str:
        return (f"新增 {len(self.new)} / 已解决 {self.resolved_count} / "
                f"持续 {len(self.persisting)}")
def diff_baseline(old_fps: set[str], alerts: list[Alert]) -> BaselineDiff:
    """基线差分：指纹在基线中为持续，不在为新增；旧指纹未命中即已解决。
    仅新增告警需要进入 L4 研判（方案 3.3.6）。
    """
    new: list[Alert] = []
    persisting: list[Alert] = []
    for a in alerts:
        (persisting if a.ensure_fingerprint() in old_fps else new).append(a)
    resolved = max(0, len(old_fps) - len(persisting))
    return BaselineDiff(new=new, persisting=persisting, resolved_count=resolved)
/* ==== ecdet/l1_build/__init__.py ==== */
"""L1 编译环境复现层 —— 解决 P1。
技术路线（方案 3.3.1）：
  1. CMake 工程 → cmake -DCMAKE_EXPORT_COMPILE_COMMANDS=ON 原生导出；
  2. Make 工程 → Bear 构建拦截产出编译数据库；
  3. Keil(.uvprojx) / IAR(.ewp) 闭源 IDE 工程 → 自研 XML 解析器，
     将包含路径、宏定义、目标核翻译为等价 clang 交叉驱动命令行；
  4. 无构建系统 → 源码目录直接扫描合成编译命令（零人工干预兜底）。
"""
from ecdet.l1_build.reconstructor import (
```

```
    BuildReconResult,
    CompileCommand,
    reconstruct,
)
__all__ = ["BuildReconResult", "CompileCommand", "reconstruct"]
/* ==== ecdet/l1_build/keil.py ==== */
"""Keil uVision 工程解析器 —— .uvprojx（XML）→ 编译要素。
提取：源文件列表、包含路径（分号分隔）、宏定义（逗号分隔）、
目标芯片提示。翻译为 clang 交叉命令行由 reconstructor 完成。
"""
from __future__ import annotations
import xml.etree.ElementTree as ET
from dataclasses import dataclass, field
from pathlib import Path
@dataclass
class KeilProject:
    target_name: str = ""
    device_hint: str = ""
    target_hint: str = ""            # 综合芯片提示（用于报告与 L2 语义标注）
    source_files: list[str] = field(default_factory=list)
    include_paths: list[str] = field(default_factory=list)
    defines: list[str] = field(default_factory=list)
def parse_uvprojx(path: str | Path) -> KeilProject:
    """解析 .uvprojx。返回 KeilProject；结构异常时抛出异常由调用方降级。"""
    root = ET.parse(str(path)).getroot()
    proj = KeilProject()
    target = root.find(".//Target")
    if target is not None:
        tn = target.find("./TargetName")
        proj.target_name = (tn.text or "").strip() if tn is not None else ""
        device = root.find(".//Device")
        if device is not None and device.text:
            proj.device_hint = device.text.strip()
    proj.target_hint = proj.device_hint or proj.target_name
    # C 编译器设置：Cads/VariousControls/{IncludePath, Define}
    vc = root.find(".//Cads/VariousControls")
    if vc is not None:
        inc = vc.find("./IncludePath")
        if inc is not None and inc.text:
            proj.include_paths = [p.strip() for p in inc.text.split(";") if p.strip()]
        df = vc.find("./Define")
        if df is not None and df.text:
            proj.defines = [d.strip() for d in df.text.split(",") if d.strip()]
    # 文件列表：Files/File/{FileName, FileType, FilePath}
    for f in root.iter("File"):
        ftype = f.find("./FileType")
        name = f.find("./FileName")
        fpath = f.find("./FilePath")
        if ftype is None or name is None or fpath is None:
            continue
```

```
        if (ftype.text or "").strip() == "1":     # 1 = C 源文件
            proj.source_files.append((fpath.text or "").strip())
    return proj
/* ==== ecdet/l1_build/iar.py ==== */
"""IAR EWARM 工程解析器 —— .ewp（XML）→ 编译要素。
IAR 工程结构（跨版本容错）：
  configuration > settings > ICCARM:
    CCDefines/name*            宏定义
    CCIncludePath2/state*      包含路径
  group > file > name          源文件（相对工程目录路径）
"""
from __future__ import annotations
import xml.etree.ElementTree as ET
from dataclasses import dataclass, field
from pathlib import Path
@dataclass
class IarProject:
    configuration: str = ""
    target_hint: str = ""
    source_files: list[str] = field(default_factory=list)
    include_paths: list[str] = field(default_factory=list)
    defines: list[str] = field(default_factory=list)
def parse_ewp(path: str | Path) -> IarProject:
    root = ET.parse(str(path)).getroot()
    proj = IarProject()
    conf = root.find(".//configuration")
    if conf is not None:
        proj.configuration = (conf.get("name") or "").strip()
    # IAR 命名空间处理：部分版本带 xmlns，直接按 localname 匹配
    for el in root.iter():
        ln = el.tag.split("}")[-1]
        if ln == "CCDefines":
            for d in el:
                if d.tag.split("}")[-1] == "name" and (d.text or "").strip():
                    proj.defines.append(d.text.strip())
        elif ln == "CCIncludePath2":
            for s in el:
                if s.tag.split("}")[-1] == "state" and (s.text or "").strip():
                    proj.include_paths.append(s.text.strip())
        elif ln == "OGChipSelectEditMenu":   # 芯片型号（如 STM32F407VG）
            for s in el:
                state = (s.text or "").strip()
                if s.tag.split("}")[-1] == "state" and state and not proj.target_hint:
                    proj.target_hint = state.split()[0]
    if not proj.target_hint:
        proj.target_hint = proj.configuration
    # 源文件：group/file/name（.c 结尾）
    for el in root.iter():
        if el.tag.split("}")[-1] == "file":
            for child in el:
```

```
                if child.tag.split("}")[-1] == "name":
                    name = (child.text or "").strip()
                    if name.lower().endswith(".c"):
                        proj.source_files.append(name.replace("\\", "/"))
    return proj
/* ==== ecdet/l1_build/reconstructor.py ==== */
"""构建复现主流程 —— BuildReconstruct（方案 3.3.1 伪代码的工程化实现）。
procedure BuildReconstruct(repo, target):
    if has CMakeLists.txt : cmake -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
    elif has Makefile     : bear -- make        # 构建拦截
    elif has *.ewp|*.uvprojx : IdeProjectParser(xml)
    else : directory-scan（源码直扫兜底）
"""
from __future__ import annotations
import json
import shutil
import subprocess
from dataclasses import dataclass, field
from pathlib import Path
from ecdet.l1_build.iar import parse_ewp
from ecdet.l1_build.keil import parse_uvprojx
# Keil/IAR 工程产出等价 clang 交叉命令行时使用的驱动前端
_CROSS_DRIVER = "clang"
_TARGET_TRIPLE_DEFAULT = "arm-none-eabi"
@dataclass
class CompileCommand:
    """compile_commands.json 单条编译命令。"""
    directory: str
    file: str
    arguments: list[str] = field(default_factory=list)
    output: str = ""
    def to_dict(self) -> dict:
        return {
            "directory": self.directory,
            "file": self.file,
            "arguments": self.arguments,
            "output": self.output,
        }
@dataclass
class BuildReconResult:
    method: str                      # cmake / bear / keil-xml / iar-xml / makefile-scan / directory-scan
    commands: list[CompileCommand] = field(default_factory=list)
    ccdb_path: Path | None = None
    warnings: list[str] = field(default_factory=list)
    target_hint: str = ""            # 目标平台提示（Keil/IAR 工程中的芯片型号）
def _carry_warnings(target: BuildReconResult, source: BuildReconResult) -> BuildReconResult:
    """降级时保留上一通道累积的过程警告。"""
    target.warnings = source.warnings + target.warnings
    return target
def reconstruct(project_root: str | Path, build_cmd: str | None = None,
```

```
                out_dir: str | Path | None = None) -> BuildReconResult:
    """入口：识别工程类型并产出 compile_commands.json。"""
    root = Path(project_root).resolve()
    out = Path(out_dir) if out_dir else root / "ecdet_build"
    out.mkdir(parents=True, exist_ok=True)
    if build_cmd:
        return _via_intercept(root, out, build_cmd)
    if (root / "CMakeLists.txt").exists():
        return _via_cmake(root, out)
    if (root / "Makefile").exists() or (root / "makefile").exists():
        return _via_makefile(root, out)
    uvprojx = sorted(root.rglob("*.uvprojx"))
    if uvprojx:
        return _via_uvprojx(root, out, uvprojx[0])
    ewp = sorted(root.rglob("*.ewp"))
    if ewp:
        return _via_ewp(root, out, ewp[0])
    return _via_directory_scan(root, out)
# ---------------------------------------------------------------------------
# 通道 1：CMake 原生导出
# ---------------------------------------------------------------------------
def _via_cmake(root: Path, out: Path) -> BuildReconResult:
    res = BuildReconResult(method="cmake")
    if shutil.which("cmake") is None:
        res.warnings.append("检测到 CMakeLists.txt 但本机未安装 cmake，退化为源码直扫")
        return _carry_warnings(_via_directory_scan(root, out), res)
    proc = subprocess.run(
        ["cmake", "-S", str(root), "-B", str(out / "cmake"),
         "-DCMAKE_EXPORT_COMPILE_COMMANDS=ON"],
        capture_output=True, text=True, timeout=600,
        encoding="utf-8", errors="replace",
    )
    ccdb = out / "cmake" / "compile_commands.json"
    if proc.returncode != 0 or not ccdb.exists():
        res.warnings.append(
            f"cmake 配置失败（退出码 {proc.returncode}），退化为源码直扫：\n"
            f"{(proc.stderr or '')[-500:]}")
        return _via_directory_scan(root, out)
    return _finish_from_ccdb(ccdb, "cmake", res)
# ---------------------------------------------------------------------------
# 通道 2：Bear 构建拦截 / 自定义构建命令拦截
# ---------------------------------------------------------------------------
def _via_intercept(root: Path, out: Path, build_cmd: str) -> BuildReconResult:
    res = BuildReconResult(method=f"intercept:{build_cmd}")
    bear = shutil.which("bear")
    if bear is None:
        res.warnings.append("本机未安装 bear，无法拦截构建，退化为源码直扫")
        return _carry_warnings(_via_directory_scan(root, out), res)
    proc = subprocess.run(
        ["bear", "--", *build_cmd.split()],
```

```
        cwd=str(root), capture_output=True, text=True, timeout=1800,
        encoding="utf-8", errors="replace",
    )
    ccdb = root / "compile_commands.json"
    if proc.returncode != 0 or not ccdb.exists():
        res.warnings.append(f"bear 拦截失败（退出码 {proc.returncode}），退化为源码直扫")
        return _carry_warnings(_via_directory_scan(root, out), res)
    return _finish_from_ccdb(ccdb, res.method, res)
# ---------------------------------------------------------------------------
# 通道 3：Makefile（bear 优先，缺 bear 则扫描 Makefile 变量）
# ---------------------------------------------------------------------------
def _via_makefile(root: Path, out: Path) -> BuildReconResult:
    res = BuildReconResult(method="makefile-scan")
    if shutil.which("bear") is not None and shutil.which("make") is not None:
        intercepted = _via_intercept(root, out, "make")
        if intercepted.commands:
            return intercepted
        res.warnings.append("bear+make 未产出编译数据库，改用 Makefile 静态解析")
    include_flags, define_flags = _scan_makefile_flags(root)
    return _synthesize_from_sources(root, out, res, include_flags, define_flags)
def _scan_makefile_flags(root: Path) -> tuple[list[str], list[str]]:
    mk = root / "Makefile"
    if not mk.exists():
        mk = root / "makefile"
    text = mk.read_text(encoding="utf-8", errors="replace")
    includes: list[str] = []
    defines: list[str] = []
    import re
    for m in re.finditer(r"-I\s*([^\s]+)", text):
        p = m.group(1).strip("'\"")
        includes.append((root / p).resolve().as_posix() if not Path(p).is_absolute() else p)
    for m in re.finditer(r"-D\s*([A-Za-z_][\w]*)", text):
        defines.append(m.group(1))
    return includes, defines
# ---------------------------------------------------------------------------
# 通道 4：Keil / IAR 工程 XML 解析
# ---------------------------------------------------------------------------
def _via_uvprojx(root: Path, out: Path, proj: Path) -> BuildReconResult:
    res = BuildReconResult(method="keil-xml")
    try:
        parsed = parse_uvprojx(proj)
    except Exception as e:  # noqa: BLE001 —— XML 格式各异，解析失败降级
        res.warnings.append(f"Keil 工程解析失败（{e}），退化为源码直扫")
        return _carry_warnings(_via_directory_scan(root, out), res)
    res.target_hint = parsed.target_hint
    src_dir = proj.parent
    includes = [(src_dir / p).resolve().as_posix() for p in parsed.include_paths]
    defines = [f"-D{d}" for d in parsed.defines]
    return _synthesize_from_sources(
        root, out, res, [f"-I{p}" for p in includes], defines,
```

```
        sources=parsed.source_files, base_dir=src_dir,
    )
def _via_ewp(root: Path, out: Path, proj: Path) -> BuildReconResult:
    res = BuildReconResult(method="iar-xml")
    try:
        parsed = parse_ewp(proj)
    except Exception as e:  # noqa: BLE001
        res.warnings.append(f"IAR 工程解析失败（{e}），退化为源码直扫")
        return _carry_warnings(_via_directory_scan(root, out), res)
    res.target_hint = parsed.target_hint
    src_dir = proj.parent
    includes = [(src_dir / p).resolve().as_posix() for p in parsed.include_paths]
    defines = [f"-D{d}" for d in parsed.defines]
    return _synthesize_from_sources(
        root, out, res, [f"-I{p}" for p in includes], defines,
        sources=parsed.source_files, base_dir=src_dir,
    )
# ---------------------------------------------------------------------------
# 通道 5：源码目录直扫（零人工干预兜底）
# ---------------------------------------------------------------------------
_SKIP_DIRS = {".git", ".svn", "build", "ecdet_build", "node_modules", "__pycache__"}
def _via_directory_scan(root: Path, out: Path) -> BuildReconResult:
    res = BuildReconResult(method="directory-scan")
    # 以根目录与各源文件所在目录为包含路径，可覆盖绝大多数小型裸机工程
    includes = [f"-I{root.resolve().as_posix()}"]
    src_dirs = {p.parent.resolve().as_posix() for p in _iter_sources(root)}
    includes += [f"-I{d}" for d in sorted(src_dirs)]
    return _synthesize_from_sources(root, out, res, includes, [])
def _iter_sources(root: Path):
    for p in root.rglob("*"):
        if p.suffix.lower() == ".c" and not any(part in _SKIP_DIRS for part in p.parts):
            yield p
def _synthesize_from_sources(root: Path, out: Path, res: BuildReconResult,
                             include_flags: list[str], define_flags: list[str],
                             sources: list[str] | None = None,
                             base_dir: Path | None = None) -> BuildReconResult:
    base = (base_dir or root).resolve()
    files = sources or [p.relative_to(root).as_posix() for p in _iter_sources(root)]
    for rel in files:
        abs_file = (base / rel).resolve()
        file_arg = abs_file.as_posix()
        args = ([_CROSS_DRIVER, f"--target={_TARGET_TRIPLE_DEFAULT}"]
                + include_flags + define_flags + ["-c", file_arg])
        res.commands.append(
            CompileCommand(directory=base.as_posix(),
                           file=file_arg, arguments=args, output="")
        )
    if not res.commands:
        res.warnings.append("未发现任何 C 源文件，产出空编译数据库")
    return _write_ccdb(out, res)
```

```
# ---------------------------------------------------------------------------
# 收尾：写出 compile_commands.json
# ---------------------------------------------------------------------------
def _finish_from_ccdb(ccdb: Path, method: str, res: BuildReconResult) -> BuildReconResult:
    res.method = method
    try:
        entries = json.loads(ccdb.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError) as e:
        res.warnings.append(f"编译数据库读取失败：{e}")
        return res
    for e in entries:
        args = e.get("arguments")
        if args is None and e.get("command"):
            args = e["command"].split()
        res.commands.append(
            CompileCommand(directory=e.get("directory", ""),
                           file=e.get("file", ""),
                           arguments=args or [],
                           output=e.get("output", ""))
        )
    return _write_ccdb(ccdb.parent, res)
def _write_ccdb(out: Path, res: BuildReconResult) -> BuildReconResult:
    ccdb = out / "compile_commands.json"
    ccdb.write_text(
        json.dumps([c.to_dict() for c in res.commands],
                   ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
    res.ccdb_path = ccdb
    return res
/* ==== ecdet/l2_ir/__init__.py ==== */
"""L2 统一 IR / 程序表示层：告警数据模型与 SARIF 交换格式。"""
from ecdet.l2_ir.model import Alert, Evidence, Location, Provenance, RuleMeta, Severity
from ecdet.l2_ir.sarif import load_sarif, save_sarif
__all__ = [
    "Alert",
    "Evidence",
    "Location",
    "Provenance",
    "RuleMeta",
    "Severity",
    "save_sarif",
    "load_sarif",
]
/* ==== ecdet/l2_ir/model.py ==== */
"""统一告警数据模型 —— L2 层核心。
所有引擎（自研规则引擎、cppcheck、clang-tidy、后续抽象解释/符号执行）
的输出都规约到这里的 Alert 结构，再由 L5 层聚合去重。
设计依据：技术方案 3.2 节“将 Clang AST/IR、调用图与 SARIF 告警统一为单一数据模型”。
"""
```

```
from __future__ import annotations
import hashlib
import json
from dataclasses import asdict, dataclass, field
from enum import Enum
from pathlib import Path
from typing import Optional
class Severity(str, Enum):
    """告警严重级别，对齐 SARIF level 并映射 GB/T 30279-2020 分级口径。"""
    ERROR = "error"        # 超危/高危：确定缺陷或安全漏洞
    WARNING = "warning"    # 中危：很可能为缺陷，需评审
    NOTE = "note"          # 低危/提示：规范偏离、可维护性问题
    def to_cn_level(self) -> str:
        """映射 GB/T 30279-2020 漏洞分级（超危/高危/中危/低危）。"""
        return {"error": "高危", "warning": "中危", "note": "低危"}[self.value]
class Provenance(str, Enum):
    """可证明性分级（技术方案 3.3.7）：
    PROVEN    —— 由健全引擎（抽象解释/模型检查）数学证明的结论，阶段二引入；
    DETECTED  —— 引擎检出且经确定性证据核对（如行号/文本回链）确认；
    SUSPECTED —— 疑似告警，需 LLM 研判或人工评审。
    """
    PROVEN = "proven"
    DETECTED = "detected"
    SUSPECTED = "suspected"
@dataclass
class Location:
    """源码位置。路径统一为 POSIX 风格相对路径，保证跨平台指纹一致。"""
    path: str
    line: int = 1
    col: int = 1
    end_line: Optional[int] = None
    end_col: Optional[int] = None
    def rel(self, base: str | Path) -> "Location":
        """以 base 为根转换为相对路径（能转换时）。"""
        try:
            p = Path(self.path).resolve()
            b = Path(base).resolve()
            return Location(
                path=p.relative_to(b).as_posix(),
                line=self.line,
                col=self.col,
                end_line=self.end_line,
                end_col=self.end_col,
            )
        except (ValueError, OSError):
            return self
@dataclass
class Evidence:
    """证据链节点：每条告警必须携带可回溯证据（技术方案 3.3.5/3.3.7）。"""
    kind: str                 # code / type / macro / api / note 等
```

```
    message: str              # 证据说明（中文）
    location: Optional[Location] = None
    snippet: str = ""         # 相关源码片段（已去注释空白）
@dataclass
class Alert:
    """统一告警对象。"""
    rule_id: str                              # 如 "MISRA C:2012 15.5"、"CERT C MSC30-C"
    engine: str                               # builtin / cppcheck / clang-tidy / ...
    severity: Severity
    message: str
    location: Location
    cwe: Optional[str] = None                 # 如 "CWE-120"
    provenance: Provenance = Provenance.SUSPECTED
    evidence: list[Evidence] = field(default_factory=list)
    fingerprint: str = ""                     # 聚合去重主键，空则按内容计算
    triage: Optional[dict] = None             # L4 分诊结论（verdict/confidence/reason/…）
    def compute_fingerprint(self) -> str:
        raw = json.dumps(
            {
                "rule": self.rule_id,
                "path": self.location.path,
                "line": self.location.line,
                "msg": self.message,
            },
            ensure_ascii=False,
            sort_keys=True,
        )
        self.fingerprint = hashlib.sha256(raw.encode("utf-8")).hexdigest()[:16]
        return self.fingerprint
    def ensure_fingerprint(self) -> str:
        return self.fingerprint or self.compute_fingerprint()
    def to_dict(self) -> dict:
        d = asdict(self)
        d["severity"] = self.severity.value
        d["provenance"] = self.provenance.value
        return d
    @classmethod
    def from_dict(cls, d: dict) -> "Alert":
        loc = d["location"]
        evs = [
            Evidence(
                kind=e.get("kind", "note"),
                message=e.get("message", ""),
                location=Location(**e["location"]) if e.get("location") else None,
                snippet=e.get("snippet", ""),
            )
            for e in d.get("evidence", [])
        ]
        return cls(
            rule_id=d["rule_id"],
```

```
            engine=d["engine"],
            severity=Severity(d.get("severity", "warning")),
            message=d["message"],
            location=Location(**loc),
            cwe=d.get("cwe"),
            provenance=Provenance(d.get("provenance", "suspected")),
            evidence=evs,
            fingerprint=d.get("fingerprint", ""),
            triage=d.get("triage"),
        )
@dataclass
class RuleMeta:
    """规则元数据：驱动规则库分层启停与合规报告生成（技术方案 3.3.3）。"""
    rule_id: str                  # "MISRA C:2012 15.5"
    standard: str                 # MISRA C:2012 / CERT C / GJB 8114-2013
    category: str                 # Mandatory / Required / Advisory
    decidability: str             # syntactic / type-based / dataflow / manual
    title_zh: str                 # 中文规则描述
    cwes: list[str] = field(default_factory=list)
    enabled_by_default: bool = True
    def to_dict(self) -> dict:
        return asdict(self)
    @classmethod
    def from_dict(cls, d: dict) -> "RuleMeta":
        return cls(
            rule_id=d["rule_id"],
            standard=d["standard"],
            category=d.get("category", "Required"),
            decidability=d.get("decidability", "syntactic"),
            title_zh=d.get("title_zh", ""),
            cwes=d.get("cwes", []),
            enabled_by_default=d.get("enabled_by_default", True),
        )
/* ==== ecdet/l2_ir/sarif.py ==== */
"""SARIF 2.1.0 读写 —— 全系统唯一告警交换格式（技术方案 3.3.7）。
自研引擎与所有外部引擎适配器的输出最终都汇入 SARIF，L5 聚合层只消费 SARIF，
保证“多引擎 → 单一表示”的解耦。
"""
from __future__ import annotations
import json
from pathlib import Path
from typing import Optional
from ecdet.l2_ir.model import Alert, Evidence, Location, Provenance, RuleMeta, Severity
SARIF_SCHEMA = (
    "https://raw.githubusercontent.com/oasis-tcs/sarif-spec/master/"
    "Schemata/sarif-schema-2.1.0.json"
)
_SEVERITY_TO_LEVEL = {
    Severity.ERROR: "error",
    Severity.WARNING: "warning",
```

```
    Severity.NOTE: "note",
}
def build_sarif(alerts: list[Alert], rules: list[RuleMeta], tool_version: str = "0.1.0") -> dict:
    """将统一告警列表构建为 SARIF 2.1.0 文档对象。"""
    rule_index: dict[str, int] = {}
    sarif_rules = []
    for i, r in enumerate(rules):
        rule_index[r.rule_id] = i
        sarif_rules.append(
            {
                "id": r.rule_id,
                "name": r.rule_id.replace(":", "").replace(" ", "_"),
                "shortDescription": {"text": r.title_zh or r.rule_id},
                "properties": {
                    "standard": r.standard,
                    "category": r.category,
                    "decidability": r.decidability,
                    "cwe": r.cwes,
                },
                "defaultConfiguration": {"level": "warning", "enabled": r.enabled_by_default},
            }
        )
    results = []
    for a in alerts:
        loc = a.location
        region = {"startLine": loc.line, "startColumn": loc.col}
        if loc.end_line:
            region["endLine"] = loc.end_line
            if loc.end_col:
                region["endColumn"] = loc.end_col
        fingerprints = a.ensure_fingerprint()
        result = {
            "ruleId": a.rule_id,
            "ruleIndex": rule_index.get(a.rule_id),
            "level": _SEVERITY_TO_LEVEL[a.severity],
            "message": {"text": a.message},
            "locations": [
                {
                    "physicalLocation": {
                        "artifactLocation": {"uri": loc.path},
                        "region": region,
                    }
                }
            ],
            "partialFingerprints": {"ecdet/v1": fingerprints},
            "properties": {
                "engine": a.engine,
                "cwe": a.cwe,
                "provenance": a.provenance.value,
                "gb_level": a.severity.to_cn_level(),
```

```
                **({"triage": a.triage} if a.triage else {}),
            },
        }
        if a.evidence:
            result["properties"]["evidence"] = [
                {
                    "kind": e.kind,
                    "message": e.message,
                    "path": e.location.path if e.location else None,
                    "line": e.location.line if e.location else None,
                    "snippet": e.snippet,
                }
                for e in a.evidence
            ]
        results.append(result)
    return {
        "$schema": SARIF_SCHEMA,
        "version": "2.1.0",
        "runs": [
            {
                "tool": {
                    "driver": {
                        "name": "ecdet",
                        "version": tool_version,
                        "informationUri": "https://example.invalid/ecdet",
                        "rules": sarif_rules,
                    }
                },
                "results": results,
            }
        ],
    }
def save_sarif(alerts: list[Alert], rules: list[RuleMeta], out_path: str | Path,
               tool_version: str = "0.1.0") -> Path:
    out = Path(out_path)
    out.parent.mkdir(parents=True, exist_ok=True)
    out.write_text(
        json.dumps(build_sarif(alerts, rules, tool_version), ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
    return out
def load_sarif(path: str | Path) -> tuple[list[Alert], list[RuleMeta]]:
    """读取 SARIF（含外部工具产出），规约为统一模型。"""
    doc = json.loads(Path(path).read_text(encoding="utf-8"))
    rules_meta: dict[str, RuleMeta] = {}
    alerts: list[Alert] = []
    for run in doc.get("runs", []):
        driver = run.get("tool", {}).get("driver", {})
        engine = driver.get("name", "unknown")
        for r in driver.get("rules", []):
```

```
            props = r.get("properties", {})
            rules_meta[r["id"]] = RuleMeta(
                rule_id=r["id"],
                standard=props.get("standard", "OTHER"),
                category=props.get("category", "Required"),
                decidability=props.get("decidability", "syntactic"),
                title_zh=r.get("shortDescription", {}).get("text", ""),
                cwes=props.get("cwe", []),
            )
        for res in run.get("results", []):
            pl = res.get("locations", [{}])[0].get("physicalLocation", {})
            loc = Location(
                path=pl.get("artifactLocation", {}).get("uri", ""),
                line=pl.get("region", {}).get("startLine", 1),
                col=pl.get("region", {}).get("startColumn", 1),
                end_line=pl.get("region", {}).get("endLine"),
                end_col=pl.get("region", {}).get("endColumn"),
            )
            props = res.get("properties", {})
            evidence = [
                Evidence(kind=e.get("kind", "note"), message=e.get("message", ""),
                         location=Location(path=e["path"], line=e["line"]) if e.get("path") else None,
                         snippet=e.get("snippet", ""))
                for e in props.get("evidence", [])
            ]
            alerts.append(
                Alert(
                    rule_id=res.get("ruleId", "unknown"),
                    engine=props.get("engine", engine),
                    severity=Severity.NOTE if res.get("level") == "note"
                    else Severity.WARNING if res.get("level") == "warning" else Severity.ERROR,
                    message=res.get("message", {}).get("text", ""),
                    location=loc,
                    cwe=props.get("cwe"),
                    provenance=Provenance(props.get("provenance", "suspected")),
                    evidence=evidence,
                    fingerprint=res.get("partialFingerprints", {}).get("ecdet/v1", ""),
                    triage=props.get("triage"),
                )
            )
    return alerts, list(rules_meta.values())
/* ==== ecdet/l3_analysis/__init__.py ==== */
"""L3 多引擎分析层。"""
from ecdet.l3_analysis.engine import (
    AnalysisTarget,
    BuiltinEngine,
    Engine,
    ExternalEngineAdapter,
    run_all_engines,
)
```

```
    return build_file_facts("t.c", textwrap.dedent(src)).lex
def test_verify_cited_lines_pass():
    src = """\
        int f(void) {
            int a[4] = {0};
            a[9] = 1;
            return 0;
        }
        """
    lex = _lex_of(src)
    ok, lines = verify_cited_lines([2, 3], lex, 1, 5, 3)
    assert ok and lines == [2, 3]
def test_verify_cited_lines_out_of_range():
    lex = _lex_of("int f(void) { return 0; }\n")
    ok, _ = verify_cited_lines([100], lex, 1, 1, 1)
    assert not ok
def test_verify_cited_line_deref_star_not_comment():
    """以 * 开头的解引用行不得误判为块注释延续。"""
    src = """\
        void f(void) {
            *(volatile unsigned int *)0x40020010 = 0u;
        }
        """
    lex = _lex_of(src)
    ok, lines = verify_cited_lines([2], lex, 1, 3, 2)
    assert ok and lines == [2]
def test_verify_cited_comment_line_rejected():
    src = """\
        int f(void) {
            /* 纯注释行 */
            return 0;
        }
        """
    lex = _lex_of(src)
    ok, _ = verify_cited_lines([2], lex, 1, 4, 3)
    assert not ok            # 只引用了注释行，且未含告警行
def test_verify_cited_must_contain_alert_line():
    lex = _lex_of("int f(void) {\n    return 0;\n}\n")
    ok, _ = verify_cited_lines([2], lex, 1, 3, 1)
    assert not ok            # 缺告警行本身
def test_verify_file_range_fallback():
    """全局声明类告警（无函数上下文）按文件范围核对。"""
    src = "uint32_t g_ctrl;\nint f(void) { return 0; }\n"
    lex = _lex_of(src)
    ok, lines = verify_cited_lines([1], lex, 0, 0, 1, has_function=False)
    assert ok and lines == [1]
# ---------------- 上下文裁剪 ----------------
def _ctx_for(tmp_path: Path):
    root = EXAMPLES / "svd_demo"
    facts = []
```

```
    for p in sorted(root.rglob("*.c")):
        facts.append(build_file_facts(
            p.relative_to(root).as_posix(),
            p.read_text(encoding="utf-8", errors="replace")))
    return finalize_facts(facts)
def test_slicer_function_closure():
    ctx = _ctx_for(None)
    alert = Alert(rule_id="ECDET-MMIO-ACCESS", engine="builtin", severity=Severity.ERROR,
                  message="x", location=Location(path="src/app.c", line=48))
    s = slice_context(alert, ctx, budget_chars=8000)
    assert s.func_name == "task_poll"
    assert "task_poll" in s.function_text
    # 闭包含跨上下文元素
    rendered = s.render()
    assert any("被调函数" in e for e in s.extras) or len(s.extras) >= 0
    assert "GPIO_TypeDef" in rendered       # 宏被纳入（GPIOA 定义）
def test_slicer_budget_truncates():
    ctx = _ctx_for(None)
    alert = Alert(rule_id="ECDET-MMIO-ACCESS", engine="builtin", severity=Severity.ERROR,
                  message="x", location=Location(path="src/app.c", line=48))
    s = slice_context(alert, ctx, budget_chars=200)
    assert s.truncated or s.n_chars <= 400
def test_slicer_missing_file():
    ctx = _ctx_for(None)
    alert = Alert(rule_id="X", engine="t", severity=Severity.WARNING,
                  message="x", location=Location(path="nope/absent.c", line=1))
    s = slice_context(alert, ctx)
    assert s.truncated and "不在工程事实中" in s.extras[0]
# ---------------- 知识库检索 ----------------
def test_kb_lookup():
    kb = build_kb()
    text = kb.lookup("CERT C STR31-C", "CWE-120")
    assert "STR31-C" in text
    assert "GJB" in text
# ---------------- Mock 分诊全路径 ----------------
def _alerts_from(root: Path):
    """对示例工程直接跑内置引擎获取真实告警。"""
    from ecdet.l3_analysis.engine import AnalysisTarget, BuiltinEngine
    from ecdet.l3_analysis.builtin_rules.rules_meta import BUILTIN_RULES
    files = [p.relative_to(root) for p in sorted(root.rglob("*.c"))]
    target = AnalysisTarget(root=root, files=files)
    return BuiltinEngine().run(target, {r.rule_id for r in BUILTIN_RULES})
def test_triage_mock_full_pipeline():
    alerts = _alerts_from(EXAMPLES / "svd_demo")
    verdicts, calls = triage_alerts(alerts, EXAMPLES / "svd_demo",
                                    client=MockLlmClient())
    by_v = {}
    for v in verdicts:
        by_v.setdefault(v.verdict, []).append(v)
    assert "TRUE_POSITIVE" in by_v and "FALSE_POSITIVE" in by_v
```

```
    # RAG 路径被触发过（低置信告警）
    assert any(v.used_rag for v in verdicts)
    # 全部回写告警
    assert all(a.triage is not None for a in alerts)
    assert calls >= len(verdicts)
def test_triage_downgrade_on_bad_citation():
    """FIO30（Mock 引用错行）→ 证据核对不通过 → TP 降级 NEEDS_REVIEW。"""
    alerts = [a for a in _alerts_from(EXAMPLES / "demo_rtos")
              if a.rule_id == "CERT C FIO30-C"]
    assert alerts, "demo_rtos 应含 FIO30-C 告警"
    verdicts, _ = triage_alerts(alerts, EXAMPLES / "demo_rtos",
                                client=MockLlmClient())
    assert all(v.verdict == "NEEDS_REVIEW" for v in verdicts)
    assert all("核对未通过" in v.reason for v in verdicts)
def test_triage_false_positive_path():
    alerts = [a for a in _alerts_from(EXAMPLES / "svd_demo")
              if a.rule_id == "MISRA C:2012 8.7"]
    assert alerts
    verdicts, _ = triage_alerts(alerts, EXAMPLES / "svd_demo",
                                client=MockLlmClient())
    assert all(v.verdict == "FALSE_POSITIVE" for v in verdicts)
def test_triage_missing_context_file():
    alerts = [Alert(rule_id="X", engine="t", severity=Severity.WARNING,
                    message="m", location=Location(path="ghost/missing.c", line=5))]
    verdicts, calls = triage_alerts(alerts, EXAMPLES / "svd_demo",
                                    client=MockLlmClient())
    assert verdicts[0].verdict == "NEEDS_REVIEW"
    assert calls == 0          # 未调用 LLM
# ---------------- SARIF 往返携带分诊 ----------------
def test_sarif_roundtrip_with_triage(tmp_path):
    from ecdet.l2_ir import load_sarif, save_sarif
    from ecdet.l3_analysis.builtin_rules.rules_meta import BUILTIN_RULES
    alerts = _alerts_from(EXAMPLES / "svd_demo")[:3]
    triage_alerts(alerts, EXAMPLES / "svd_demo", client=MockLlmClient())
    path = save_sarif(alerts, list(BUILTIN_RULES), tmp_path / "t.sarif")
    loaded, _ = load_sarif(path)
    assert all(a.triage is not None for a in loaded)
    assert loaded[0].triage["verdict"] in ("TRUE_POSITIVE", "FALSE_POSITIVE",
                                           "NEEDS_REVIEW")
/* ==== tests/test_dynamic_agents.py ==== */
# -*- coding: utf-8 -*-
"""V1.2 对齐测试：动态通道（3.3.5）、运行时监控（3.3.6）、多智能体编排（3.3.7）。"""
import textwrap
from pathlib import Path
import pytest
from ecdet.l2_ir.model import Alert, Location, Provenance, Severity
from ecdet.l3_analysis.builtin_rules.checks import build_file_facts, finalize_facts
from ecdet.l3_analysis.dynamic import DynamicEngine
from ecdet.l3_analysis.dynamic.harness import compile_harness, generate_harness
from ecdet.l3_analysis.dynamic.inputs import _INT_SPECIALS, build_input_model, param_value
```

```
from ecdet.l3_analysis.monitor import RuntimeMonitor, probes_from_alerts
from ecdet.l4_llm.agents import Orchestrator, OrchestratorConfig
pytestmark = pytest.mark.skipif(
    DynamicEngine().available is False, reason="需要 gcc")
DYN_C = textwrap.dedent("""\
    #include <stdint.h>
    int scale_value(int base, int divisor)
    {
        return base / divisor;
    }
    int read_selected(char *buf, int sel)
    {
        const char *p = (sel > 100) ? 0 : buf;
        return p[0];
    }
    int safe_add(int a, int b) { return a + b; }
    """)
@pytest.fixture()
def dyn_root(tmp_path):
    (tmp_path / "vuln_dyn.c").write_text(DYN_C, encoding="utf-8")
    return tmp_path
def _facts(root: Path):
    return [build_file_facts("vuln_dyn.c",
                             (root / "vuln_dyn.c").read_text(encoding="utf-8"))]
# ---------------- 输入建模 ----------------
def test_input_model_types_from_parser(dyn_root):
    facts = _facts(dyn_root)
    ctx = finalize_facts(facts)
    func = next(f for f in ctx.facts[0].functions if f.name == "read_selected")
    model = build_input_model(func)
    buf = next(p for p in model.params if p.name == "buf")
    sel = next(p for p in model.params if p.name == "sel")
    assert buf.is_pointer, "cstruct 解析的参数类型应为 char *"
    assert not sel.is_pointer and sel.ctype == "int"
def test_param_specials_cover_zero():
    """特殊值表含 0 —— 除零 PoC 的确定性来源。"""
    assert 0 in _INT_SPECIALS
    from ecdet.l3_analysis.dynamic.inputs import ParamSpec
    spec = ParamSpec(name="divisor", ctype="int")
    assert param_value(spec, 0).lstrip("-").isdigit()
# ---------------- harness 生成与编译闭环 ----------------
def test_harness_generate_and_compile(dyn_root, tmp_path):
    facts = _facts(dyn_root)
    ctx = finalize_facts(facts)
    func = next(f for f in ctx.facts[0].functions if f.name == "scale_value")
    model = build_input_model(func)
    harness = generate_harness(model, "vuln_dyn.c", n_seeds=17)
    assert "ecdet_combo" in harness.source
    assert '#include "vuln_dyn.c"' in harness.source
    ok, err = compile_harness(harness, tmp_path, [dyn_root])
```

```
    assert ok, err
    assert harness.exe_path and harness.exe_path.exists()
def test_harness_stub_loop_for_undefined_symbols(tmp_path):
    """链接期未解析符号 → 自动打桩重编（生成-编译-修正闭环）。"""
    src = textwrap.dedent("""\
        int target_fn(int x) { return helper_one(x) + helper_two(x); }
        """)
    (tmp_path / "stub_target.c").write_text(src, encoding="utf-8")
    facts = [build_file_facts("stub_target.c", src)]
    ctx = finalize_facts(facts)
    func = ctx.facts[0].functions[0]
    model = build_input_model(func)
    harness = generate_harness(model, "stub_target.c", n_seeds=4)
    ok, err = compile_harness(harness, tmp_path, [tmp_path])
    assert ok, err
    assert set(harness.stubs) == {"helper_one", "helper_two"}
def test_poc_replay_seed_consistent(dyn_root):
    """PoC 重放种子与激励索引一致（可复现性契约）。"""
    findings = DynamicEngine(max_functions=8, max_seeds=17).run(
        dyn_root, _facts(dyn_root))
    div = next(f for f in findings if f.func_name == "scale_value")
    assert div.oracle == "CRASH" and "C0000094" in div.detail
    assert div.replay_seed == str(div.seed_idx % 17)
# ---------------- 动态引擎 ----------------
def test_dynamic_engine_triggers(dyn_root):
    findings = DynamicEngine(max_functions=8, max_seeds=17).run(
        dyn_root, _facts(dyn_root))
    by_func = {f.func_name: f for f in findings}
    assert by_func["scale_value"].oracle == "CRASH"
    assert "C0000094" in by_func["scale_value"].detail       # 整数除零
    assert by_func["read_selected"].oracle == "CRASH"
    assert "C0000005" in by_func["read_selected"].detail     # 空指针
    assert "safe_add" not in by_func, "干净函数不得误触发"
    alert = by_func["scale_value"].to_alert()
    assert alert.engine == "dynamic"
    assert alert.provenance == Provenance.DETECTED
    assert alert.location.line >= 3, "告警应定位在函数定义行"
def test_dynamic_alerts_in_function_range(dyn_root):
    """动态告警行必须落在目标函数行区间内（函数级评测可覆盖）。"""
    facts = _facts(dyn_root)
    findings = DynamicEngine(max_functions=8, max_seeds=17).run(dyn_root, facts)
    ctx = finalize_facts(facts)
    for f in findings:
        func = next(fn for fn in ctx.facts[0].functions
                    if fn.name == f.func_name)
        assert func.def_line <= f.def_line <= func.body_close_line
# ---------------- 运行时监控（3.3.6） ----------------
def test_monitor_probes_and_regrade(dyn_root):
    """Suspected 级嫌疑 → 定向探针 → 崩溃命中 → 升级 Detected。"""
    facts = _facts(dyn_root)
```

```
    ctx = finalize_facts(facts)
    func = next(f for f in ctx.facts[0].functions if f.name == "scale_value")
    suspected = Alert(
        rule_id="CERT C ARR30-C", engine="builtin-absint",
        severity=Severity.WARNING,
        message="疑似越界路径", location=Location(path="vuln_dyn.c",
                                                  line=func.def_line),
        provenance=Provenance.SUSPECTED)
    probes = probes_from_alerts([suspected], facts)
    assert probes == ["scale_value"]
    result = RuntimeMonitor(
        engine=DynamicEngine(max_functions=8, max_seeds=17)).run(
        dyn_root, facts, [suspected])
    assert result.events, "定向探针应命中崩溃"
    assert suspected in result.upgraded
    assert suspected.provenance == Provenance.DETECTED
    assert any("运行时监控升级" in e.message for e in suspected.evidence)
def test_monitor_no_probe_skipped(dyn_root):
    facts = _facts(dyn_root)
    result = RuntimeMonitor().run(dyn_root, facts, [])
    assert result.events == []
    assert "无可动态裁决" in result.skipped_reason
# ---------------- 多智能体编排（3.3.7） ----------------
EVAL_CORPUS = Path(__file__).parent / "fixtures" / "eval_corpus"
def test_orchestrate_multi_agent_full_pipeline():
    state = Orchestrator(OrchestratorConfig(mock=True)).orchestrate(EVAL_CORPUS)
    agents_logged = {line.split("]")[0].lstrip("[") for line in state.logs}
    assert {"planner", "static", "static-triage", "dynamic",
            "reporter"} <= agents_logged
    dyn_alerts = [a for a in state.static_alerts if a.engine == "dynamic"]
    assert len(dyn_alerts) >= 2, "动态通道 PoC 应入 SARIF 池"
    assert state.llm_calls > 0
    assert all(a.triage is not None for a in state.static_alerts
               if a.engine != "dynamic" and a.severity.value != "note")
def test_orchestrate_single_agent_parity():
    """A6：单智能体串行产出与多智能体一致（组织学消融，检测能力等价）。"""
    multi = Orchestrator(OrchestratorConfig(mock=True)).orchestrate(EVAL_CORPUS)
    single = Orchestrator(OrchestratorConfig(mock=True,
                                             single_agent=True)).orchestrate(
        EVAL_CORPUS)
    assert len(multi.static_alerts) == len(single.static_alerts)
    assert len(multi.dynamic_findings) == len(single.dynamic_findings)
def test_orchestrate_a1_no_llm():
    state = Orchestrator(OrchestratorConfig(mock=True, use_llm=False)
                         ).orchestrate(EVAL_CORPUS)
    assert state.triage_verdicts == []
    assert all(a.triage is None for a in state.static_alerts
               if a.engine != "dynamic")
/* ==== tests/test_dynamic_fixers.py ==== */
# -*- coding: utf-8 -*-
```

```
"""动态通道补强测试：LLM harness 修正闭环（3.3.5）与三级载体梯度。"""
import textwrap
from pathlib import Path
import pytest
from ecdet.l3_analysis.builtin_rules.checks import build_file_facts, finalize_facts
from ecdet.l3_analysis.dynamic import DynamicEngine
from ecdet.l3_analysis.dynamic.carriers import (
    HostCarrier,
    RenodeCarrier,
    UnicornCarrier,
    pick_carrier,
    probe_carriers,
)
from ecdet.l3_analysis.dynamic.harness import compile_harness, generate_harness
from ecdet.l3_analysis.dynamic.inputs import build_input_model
from ecdet.l3_analysis.dynamic.llm_fix import (
    LlmHarnessFixer,
    extract_patched_harness,
)
pytestmark = pytest.mark.skipif(not HostCarrier.available(), reason="需要 gcc")
# ---------------- extract_patched_harness ----------------
def test_extract_patched_harness_plain():
    raw = '{"patched_harness": "int main(void){return 0;}", "changes": "补 include"}'
    got = extract_patched_harness(raw)
    assert got == ("int main(void){return 0;}", "补 include")
def test_extract_patched_harness_fence():
    raw = '```json\n{"patched_harness": "```c\\nint x;\\n```", "changes": ""}\n```'
    got = extract_patched_harness(raw)
    assert got is not None and got[0] == "int x;"
def test_extract_patched_harness_invalid():
    assert extract_patched_harness("不会输出 JSON") is None
    assert extract_patched_harness('{"patched_harness": ""}') is None
    assert extract_patched_harness("") is None
# ---------------- LLM 修正闭环 ----------------
BROKEN_C = textwrap.dedent("""\
    #include <stdint.h>
    int broken_fn(int mode)
    {
        undefined_helper_abc(mode);      /* 未定义符号：打桩可修 */
        return (unknown_type_xyz *) 0;   /* 类型不存在：打桩不可修，需 LLM */
    }
    """)
def test_llm_fixer_recovers_uncompilable_harness(tmp_path):
    """确定性打桩失败 → LLM 修正（Fake：修复坏行）→ 编译通过。
    直接验证 compile_harness 的"编译日志回传 → LLM 重写 → 再编译"
    闭环机制（方案 3.3.5）。
    """
    good_c = textwrap.dedent("""\
        #include <stdint.h>
        int simple_fn(int mode) { return mode * 2; }
```

```
        """)
    (tmp_path / "simple.c").write_text(good_c, encoding="utf-8")
    facts = [build_file_facts("simple.c", good_c)]
    ctx = finalize_facts(facts)
    func = next(f for f in ctx.facts[0].functions if f.name == "simple_fn")
    model = build_input_model(func)
    harness = generate_harness(model, "simple.c", n_seeds=2)
    class FakeLlm:
        """确定性 Fake：收到含坏行的源码即删除该行。"""
        def __init__(self):
            self.calls = 0
        def complete(self, system, user):
            self.calls += 1
            import json
            import re
            fm = re.search(r"```c\n(.*?)\n```", user, re.S)
            src_text = fm.group(1) if fm else ""
            fixed = "\n".join(ln for ln in src_text.splitlines()
                              if "ecdet_deliberate_broken_line" not in ln)
            return json.dumps({"patched_harness": fixed,
                               "changes": "删除语法坏行"},
                              ensure_ascii=False)
    # 注入一条真正的语法错误（初始化缺表达式；无打桩模式可修）
    harness.source = harness.source.replace(
        "static const int",
        "int ecdet_deliberate_broken_line = ;\nstatic const int")
    fixer = LlmHarnessFixer(FakeLlm())
    ok, err = compile_harness(harness, tmp_path, [tmp_path], fixer=fixer)
    assert ok, err
    assert fixer.calls >= 1, "LLM 修正闭环应被触发"
    assert "ecdet_deliberate_broken_line" not in harness.source
def test_no_fixer_fails_gracefully(tmp_path):
    """无 fixer 且打桩不可修复 → 明确失败（不挂起）。"""
    (tmp_path / "broken.c").write_text(BROKEN_C, encoding="utf-8")
    facts = [build_file_facts("broken.c", BROKEN_C)]
    ctx = finalize_facts(facts)
    func = next(f for f in ctx.facts[0].functions if f.name == "broken_fn")
    model = build_input_model(func)
    harness = generate_harness(model, "broken.c", n_seeds=2)
    ok, err = compile_harness(harness, tmp_path, [tmp_path])
    assert not ok and err
def test_engine_with_fixer_still_finds_crashes(tmp_path):
    """修正钩子不影响正常路径：除零 PoC 照常触发。"""
    src = textwrap.dedent("""\
        #include <stdint.h>
        int scale_value(int base, int divisor) { return base / divisor; }
        """)
    class NoopClient:
        """编译一次通过时不应被调用的客户端探针。"""
        calls = 0
```

```
        def complete(self, system, user):
            NoopClient.calls += 1
            return "不输出有效 JSON"
    (tmp_path / "m.c").write_text(src, encoding="utf-8")
    facts = [build_file_facts("m.c", src)]
    engine = DynamicEngine(max_functions=4, max_seeds=17,
                           fixer=LlmHarnessFixer(NoopClient()))
    findings = engine.run(tmp_path, facts)
    assert any(f.func_name == "scale_value" for f in findings)
    assert NoopClient.calls == 0, "编译一次通过时不得触发 LLM 修正"
# ---------------- 三级载体梯度 ----------------
def test_carrier_probe_reports_all_tiers():
    infos = {c.name: c for c in probe_carriers()}
    assert set(infos) == {"host", "unicorn", "renode"}
    assert infos["host"].tier == 1 and infos["host"].available
    assert infos["unicorn"].tier == 2 and not infos["unicorn"].available
    assert "unicorn" in infos["unicorn"].note
    assert infos["renode"].tier == 3 and not infos["renode"].available
def test_pick_carrier_prefers_highest_throughput():
    carrier, info = pick_carrier()
    assert carrier is HostCarrier and info.available
def test_pick_carrier_explicit_unavailable():
    carrier, info = pick_carrier("renode")
    assert carrier is None and info.available is False
/* ==== tests/test_fixer.py ==== */
# -*- coding: utf-8 -*-
"""阶段三后半：LLM 修复建议与 ToolLoopVerify 验证闭环测试。"""
import json
import textwrap
from pathlib import Path
from ecdet.l2_ir.model import Alert, Location, Severity
from ecdet.l3_analysis.builtin_rules.checks import build_file_facts, finalize_facts
from ecdet.l4_llm.fixer import (
    FixProposer,
    FixVerifier,
    MockFixClient,
    apply_patch,
    extract_patched_function,
)
EXAMPLES = Path(__file__).parent.parent / "examples"
# ---------------- 补丁应用与解析 ----------------
def test_apply_patch_replaces_region():
    src = "int a;\nint f(void) {\n    return 0;\n}\nint b;\n"
    out = apply_patch(src, 2, 4, "int f(void) {\n    return 1;\n}")
    assert out == "int a;\nint f(void) {\n    return 1;\n}\nint b;\n"
def test_extract_patched_function_plain():
    raw = json.dumps({"patched_function": "int f(void) { return 1; }",
                      "explanation": "改返回值"}, ensure_ascii=False)
    parsed = extract_patched_function(raw)
    assert parsed["patched_function"] == "int f(void) { return 1; }"
```

```
    assert parsed["explanation"] == "改返回值"
def test_extract_patched_function_with_fence():
    raw = '```json\n{"patched_function": "```c\\nint f(void) { return 1; }\\n```", ' \
          '"explanation": ""}\n```'
    parsed = extract_patched_function(raw)
    assert parsed["patched_function"] == "int f(void) { return 1; }"
def test_extract_patched_function_invalid():
    assert extract_patched_function("not json") == {}
    assert extract_patched_function('{"patched_function": ""}') == {}
# ---------------- 验证闭环三路径 ----------------
def _alert_for(root: Path, rule: str, path: str, line: int) -> Alert:
    return Alert(rule_id=rule, engine="builtin", severity=Severity.ERROR,
                 message="test", location=Location(path=path, line=line))
def _ctx_for(root: Path):
    facts = []
    for p in sorted(root.rglob("*.c")):
        facts.append(build_file_facts(
            p.relative_to(root).as_posix(),
            p.read_text(encoding="utf-8", errors="replace")))
    return finalize_facts(facts)
def test_fix_str31_verified(tmp_path):
    """STR31：Mock 正确补丁 → 原告警消除 + 无新 error + gcc 语法通过 → 采纳。"""
    ctx = _ctx_for(EXAMPLES / "demo_rtos")
    alert = _alert_for(EXAMPLES / "demo_rtos", "CERT C STR31-C",
                       "src/sensor.c", 37)
    proposal = FixProposer(MockFixClient()).propose(alert, ctx)
    assert proposal.patched_code, "Mock 应给出补丁"
    verifier = FixVerifier(EXAMPLES / "demo_rtos", use_compiler=True)
    src = (EXAMPLES / "demo_rtos" / "src" / "sensor.c").read_text(encoding="utf-8")
    proposal = verifier.verify(proposal, ctx, src)
    assert proposal.verified, proposal.fail_reason
    assert "strncpy" in proposal.patched_code
    assert "@@ -" in proposal.diff and "strncpy" in proposal.diff
    assert "strcpy(" not in proposal.patched_code
def test_fix_no_change_rejected():
    """Mock 无变化路径 → 拒绝（补丁未改变函数）。"""
    ctx = _ctx_for(EXAMPLES / "demo_rtos")
    alert = _alert_for(EXAMPLES / "demo_rtos", "MISRA C:2012 17.2",
                       "src/sensor.c", 28)
    proposal = FixProposer(MockFixClient()).propose(alert, ctx)
    verifier = FixVerifier(EXAMPLES / "demo_rtos", use_compiler=False)
    src = (EXAMPLES / "demo_rtos" / "src" / "sensor.c").read_text(encoding="utf-8")
    proposal = verifier.verify(proposal, ctx, src)
    assert not proposal.verified
    assert "未改变" in proposal.fail_reason
def test_fix_not_in_function_rejected():
    """全局告警（ISR-RACE 定位声明行附近但不在函数内）→ 无法生成补丁。"""
    ctx = _ctx_for(EXAMPLES / "svd_demo")
    alert = _alert_for(EXAMPLES / "svd_demo", "ECDET-ISR-RACE",
                       "src/app.c", 22)     # g_ctrl 声明行
```

```
    proposal = FixProposer(MockFixClient()).propose(alert, ctx)
    assert not proposal.verified
    assert "不在函数内" in proposal.fail_reason
def test_fix_proposer_unparsable_output():
    """LLM 输出不可解析 → 空补丁 + 失败原因，不进入验证。"""
    class BadClient:
        def complete(self, system, user):
            return "我不会输出 JSON"
    ctx = _ctx_for(EXAMPLES / "demo_rtos")
    alert = _alert_for(EXAMPLES / "demo_rtos", "CERT C STR31-C",
                       "src/sensor.c", 37)
    proposal = FixProposer(BadClient()).propose(alert, ctx)
    assert not proposal.patched_code
    assert "不可解析" in proposal.fail_reason
def test_fix_alert_gone_check(tmp_path):
    """静态重扫口径：补丁后原告警仍存在 → 拒绝（用内存语料精确控制）。"""
    src = textwrap.dedent("""\
        #include <string.h>
        void f(void) {
            char b[4];
            strcpy(b, "too long");
        }
        """)
    root = tmp_path
    (root / "m.c").write_text(src, encoding="utf-8")
    ctx = finalize_facts([build_file_facts("m.c", src)])
    # Mock 的 STR31 正确补丁把 strcpy 换成限长 strncpy → 原告警应消除
    alert = _alert_for(root, "CERT C STR31-C", "m.c", 4)
    proposal = FixProposer(MockFixClient()).propose(alert, ctx)
    verifier = FixVerifier(root, use_compiler=True)
    proposal = verifier.verify(proposal, ctx, src)
    assert proposal.verified, proposal.fail_reason
/* ==== tests/test_incremental.py ==== */
# -*- coding: utf-8 -*-
"""增量分析测试：影响集计算与告警基线差分（方案 3.3.6）。"""
import textwrap
from pathlib import Path
from ecdet.incremental import (
    BaselineDiff,
    compute_impact_set,
    diff_baseline,
    save_baseline,
    load_baseline_fingerprints,
)
from ecdet.l2_ir.model import Alert, Location, Severity
CORPUS = textwrap.dedent("""\
    #ifndef SENSOR_H
    #define SENSOR_H
    int sensor_read(void);
    #endif
```

```
    """)
MAIN_C = textwrap.dedent("""\
    #include "sensor.h"
    int main(void) { return sensor_read(); }
    """)
SENSOR_C = textwrap.dedent("""\
    #include "sensor.h"
    static int level;
    int sensor_read(void) { level++; return level; }
    """)
COMM_C = textwrap.dedent("""\
    #include <stdint.h>
    void comm_send(void) { }
    """)
def _make_corpus(root: Path) -> Path:
    (root / "sensor.h").write_text(CORPUS, encoding="utf-8")
    (root / "main.c").write_text(MAIN_C, encoding="utf-8")
    (root / "sensor.c").write_text(SENSOR_C, encoding="utf-8")
    (root / "comm.c").write_text(COMM_C, encoding="utf-8")
    return root
def test_impact_set_call_closure(tmp_path):
    """改 sensor.c → 调用其函数的 main.c 受影响；无关的 comm.c 不受影响。"""
    root = _make_corpus(tmp_path)
    impact = compute_impact_set(root, ["sensor.c"])
    assert "sensor.c" in impact.affected
    assert impact.reason["sensor.c"] == "changed"
    assert "main.c" in impact.affected
    assert "调用受影响文件的函数" in impact.reason["main.c"]
    assert "comm.c" not in impact.affected
def test_impact_set_include_closure(tmp_path):
    """改 sensor.h → 包含它的 main.c 与 sensor.c 均受影响。"""
    root = _make_corpus(tmp_path)
    impact = compute_impact_set(root, ["sensor.h"])
    assert {"sensor.h", "main.c", "sensor.c"} <= set(impact.affected)
    assert "包含受影响头文件" in impact.reason["main.c"]
    assert "comm.c" not in impact.affected
def test_impact_set_transitive_include(tmp_path):
    """include 闭包传递：a.h ← b.h ← c.c，改 a.h 三者皆受影响。"""
    (tmp_path / "a.h").write_text("#define A 1\n", encoding="utf-8")
    (tmp_path / "b.h").write_text('#include "a.h"\n#define B 2\n', encoding="utf-8")
    (tmp_path / "c.c").write_text('#include "b.h"\nint f(void) { return A + B; }\n',
                                  encoding="utf-8")
    impact = compute_impact_set(tmp_path, ["a.h"])
    assert {"a.h", "b.h", "c.c"} <= set(impact.affected)
# ---------------- 基线差分 ----------------
def _alert(rule: str, path: str, line: int, msg: str = "m") -> Alert:
    return Alert(rule_id=rule, engine="builtin", severity=Severity.WARNING,
                 message=msg, location=Location(path=path, line=line))
def test_baseline_diff_classification():
    old = {_alert("R1", "a.c", 5).compute_fingerprint(),
```

```
            _alert("R2", "a.c", 9).compute_fingerprint()}
    alerts = [
        _alert("R1", "a.c", 5),            # 持续
        _alert("R3", "b.c", 12),           # 新增
        _alert("R4", "b.c", 20),           # 新增
    ]
    diff = diff_baseline(old, alerts)
    assert isinstance(diff, BaselineDiff)
    assert len(diff.new) == 2
    assert len(diff.persisting) == 1
    assert diff.resolved_count == 1        # R2 未再出现
    assert "新增 2" in diff.summary
def test_baseline_roundtrip(tmp_path):
    alerts = [_alert("R1", "a.c", 5), _alert("R2", "b.c", 9)]
    path = save_baseline(alerts, tmp_path / "baseline.json")
    fps = load_baseline_fingerprints(path)
    assert fps == {a.ensure_fingerprint() for a in alerts}
    diff = diff_baseline(fps, alerts)
    assert diff.new == [] and diff.resolved_count == 0
    assert len(diff.persisting) == 2
/* ==== tests/test_eval.py ==== */
# -*- coding: utf-8 -*-
"""阶段四：评测框架测试（消融 A0-A3、指标数学、报告渲染）。"""
import json
from pathlib import Path
from ecdet.eval import (
    AblationConfig,
    compute_metrics,
    gt_from_corpus,
    load_ground_truth,
    render_markdown,
    run_baseline,
    run_ecdet_config,
)
from ecdet.l2_ir.model import Alert, Location, Severity
from ecdet.l3_analysis.engine import DEFAULT_ENGINES
CORPUS = Path(__file__).parent / "fixtures" / "eval_corpus"
# 竞争缺陷以无保护访问侧函数为检测目标（规则定位首个无保护访问行）；
# UART8_IRQHandler 为 ISR 侧写入方，不单独承载告警行。
VULN_FUNCS = {"buf_copy_bad", "TIM6_IRQHandler", "gpio_ro_write",
              "unprotected_reader", "scale_value", "read_selected"}
def _gt():
    return gt_from_corpus(CORPUS, VULN_FUNCS)
def test_gt_generation():
    gt = _gt()
    by_name = {g.func_name: g for g in gt}
    assert by_name["buf_copy_bad"].label == "vulnerable"
    assert by_name["buf_copy_ok"].label == "clean"
    assert by_name["utility_helper"].label == "clean"
    assert all(g.end_line >= g.start_line for g in gt)
```

```
def test_load_ground_truth_jsonl(tmp_path):
    p = tmp_path / "gt.jsonl"
    p.write_text(json.dumps({
        "path": "a.c", "func_name": "f", "start_line": 1, "end_line": 9,
        "label": "vulnerable", "cwe": "CWE-120"}) + "\n", encoding="utf-8")
    gt = load_ground_truth(p)
    assert len(gt) == 1 and gt[0].cwe == "CWE-120"
# ---------------- 指标数学 ----------------
def test_metrics_perfect():
    gt = _gt()
    run = run_ecdet_config(CORPUS, AblationConfig("A0 完整系统"),
                           svd_path=CORPUS / "chip.svd")
    m = compute_metrics(run.alerts, gt)
    assert m["func_fn"] == 0, "A0 应检出全部漏洞函数（含动态通道）"
    assert m["func_fp"] == 0, "Mock 分诊应过滤 utility_helper 的 8.7 误报"
    assert m["precision"] == 1.0 and m["recall"] == 1.0
    # 动态贡献：除零/空指针由动态通道独家检出
    assert set(m["dynamic_only_funcs"]) >= {"scale_value", "read_selected"}
    assert m["dynamic_contribution"] > 0
def test_ablation_a5_no_dynamic_misses_dyn_defects():
    """A5（去动态通道）：静态不可判定的除零/空指针漏检 → 召回下降。"""
    gt = _gt()
    run = run_ecdet_config(CORPUS, AblationConfig("A5 去动态通道",
                                                  use_dynamic=False),
                           svd_path=CORPUS / "chip.svd")
    m = compute_metrics(run.alerts, gt)
    assert "scale_value" not in m["dynamic_only_funcs"]
    assert m["func_fn"] >= 2, "去动态通道后应漏检动态专项漏洞函数"
    assert m["recall"] < 1.0
def test_ablation_a6_single_agent_runs():
    """A6（单智能体串行）：管线可完整运行，产出与消融指标一致。"""
    run = run_ecdet_config(CORPUS, AblationConfig("A6 单智能体串行",
                                                  single_agent=True),
                           svd_path=CORPUS / "chip.svd")
    assert run.alerts is not None
    m = compute_metrics(run.alerts, _gt())
    assert m["func_tp"] >= 1
def test_metrics_synthetic():
    gt = [
        dict(path="a.c", func_name="v1", start_line=1, end_line=10,
             label="vulnerable"),
        dict(path="a.c", func_name="v2", start_line=12, end_line=20,
             label="vulnerable"),
        dict(path="a.c", func_name="c1", start_line=22, end_line=30,
             label="clean"),
    ]
    def mk(line):
        return Alert(rule_id="R", engine="t", severity=Severity.WARNING,
                     message="m", location=Location(path="a.c", line=line))
    alerts = [mk(5), mk(25)]           # 命中 v1（TP）与 c1（FP），漏 v2（FN）
```

```
    m = compute_metrics(alerts, [type("G", (), d)() for d in []] or
                        [_gt_like(d) for d in gt])
    assert m["func_tp"] == 1 and m["func_fn"] == 1 and m["func_fp"] == 1
    assert abs(m["precision"] - 0.5) < 1e-6
    assert abs(m["recall"] - 0.5) < 1e-6
    assert abs(m["f1"] - 0.5) < 1e-6
    assert m["alert_tp"] == 1 and m["alert_fp"] == 1
def _gt_like(d):
    from ecdet.eval import GTEntry
    return GTEntry(path=d["path"], func_name=d["func_name"],
                   start_line=d["start_line"], end_line=d["end_line"],
                   label=d["label"])
# ---------------- 消融差异 ----------------
def test_ablation_a1_no_llm_keeps_fp():
    """A1（去 LLM）：8.7 误报保留 → FP > 0，精确率低于 A0。"""
    gt = _gt()
    run = run_ecdet_config(CORPUS, AblationConfig("A1 去 LLM", use_llm=False),
                           svd_path=CORPUS / "chip.svd")
    m = compute_metrics(run.alerts, gt)
    assert m["func_fp"] >= 1
    assert m["recall"] == 1.0           # 静态骨架仍检出全部漏洞函数
    a0 = run_ecdet_config(CORPUS, AblationConfig("A0"), svd_path=CORPUS / "chip.svd")
    m0 = compute_metrics(a0.alerts, gt)
    assert m0["precision"] > m["precision"]
def test_ablation_a2_no_hw_misses_hw_defects():
    """A2（去硬件语义）：ISR-BLOCK/RO 写漏洞漏检 → 召回下降。"""
    gt = _gt()
    run = run_ecdet_config(CORPUS, AblationConfig("A2 去硬件语义", use_llm=False,
                                                  use_hw=False))
    m = compute_metrics(run.alerts, gt)
    assert m["func_fn"] >= 1, "去掉硬件语义后至少漏检一个硬件类漏洞"
    assert m["recall"] < 1.0
    a0 = run_ecdet_config(CORPUS, AblationConfig("A0", use_llm=False),
                          svd_path=CORPUS / "chip.svd")
    m0 = compute_metrics(a0.alerts, gt)
    assert m0["recall"] > m["recall"]
def test_ablation_a3_no_rag_still_runs():
    """A3（去 RAG）：管线可运行，RAG 不触发。"""
    run = run_ecdet_config(CORPUS, AblationConfig("A3 去 RAG", use_rag=False),
                           svd_path=CORPUS / "chip.svd")
    assert run.alerts is not None
    # 阈值 0 意味着永不触发 RAG
    from ecdet.l4_llm.triage import TriageRunner
    r = TriageRunner(rag_conf_threshold=0.0)
    assert r.rag_conf_threshold == 0.0
# ---------------- 基线与报告 ----------------
def test_baseline_unavailable_marked():
    class FakeEngine:
        name = "notinstalled"
        def available(self):
```

```
            return False
        def run(self, target, enabled):
            return []
    br = run_baseline(CORPUS, FakeEngine())
    assert not br.available and br.note == "未安装，跳过"
def test_render_markdown_table():
    gt = _gt()
    run = run_ecdet_config(CORPUS, AblationConfig("A0 完整系统"),
                           svd_path=CORPUS / "chip.svd")
    md = render_markdown([(run, compute_metrics(run.alerts, gt))])
    assert "| A0 完整系统 |" in md
    assert "精确率" in md and "可证明性比例" in md
def test_builtin_engine_present_in_defaults():
    names = [e.name for e in DEFAULT_ENGINES]
    assert "builtin" in names
/* ==== tests/test_dataset.py ==== */
# -*- coding: utf-8 -*-
"""阶段三：数据集采集管线测试（离线 git 仓库 fixture）。"""
import json
import subprocess
import sys
from pathlib import Path
import pytest
sys.path.insert(0, str(Path(__file__).parent.parent / "scripts"))
from build_dataset import (  # noqa: E402
    Manifest,
    Advisory,
    build_samples,
    extract_functions,
    fetch_ghsa_advisories,
    find_fix_commits,
    fingerprint_text,
    ghsa_to_advisories,
    load_manifest,
    write_dataset,
)
pytest.importorskip("git") if False else None   # git 由 subprocess 调用
def _git(repo: Path, *args: str) -> str:
    env = {"PATH": "/usr/bin:/bin:/mingw64/bin",
           "GIT_AUTHOR_NAME": "t", "GIT_AUTHOR_EMAIL": "t@t",
           "GIT_COMMITTER_NAME": "t", "GIT_COMMITTER_EMAIL": "t@t",
           "HOME": str(repo.parent)}
    r = subprocess.run(["git", "-C", str(repo), *args], capture_output=True,
                       text=True, encoding="utf-8", env={**__import__("os").environ,
                                                         **env})
    assert r.returncode == 0, r.stderr
    return r.stdout
VULN_C_V1 = """\
#include <stdint.h>
static uint8_t g_len;
```

```
/* 拷贝传感器名称（存在溢出） */
void sensor_set_name(const char *name)
{
    char buf[8];
    uint8_t i = 0;
    while (name[i] && i < 255u) {
        buf[i] = name[i];
        i++;
    }
    g_len = i;
}
void sensor_ok(void)
{
    g_len = 0u;
}
"""
VULN_C_V2 = """\
#include <stdint.h>
static uint8_t g_len;
/* 拷贝传感器名称（修复：限长拷贝） */
void sensor_set_name(const char *name)
{
    char buf[8];
    uint8_t i = 0;
    while (name[i] && i < 7u) {
        buf[i] = name[i];
        i++;
    }
    buf[7] = 0;
    g_len = i;
}
void sensor_ok(void)
{
    g_len = 0u;
}
"""
@pytest.fixture(scope="module")
def demo_repo(tmp_path_factory):
    repo = tmp_path_factory.mktemp("rt_thread_like")
    (repo / "driver.c").write_text(VULN_C_V1, encoding="utf-8")
    _git(repo, "init", "-q")
    _git(repo, "add", "-A")
    _git(repo, "-c", "user.name=t", "-c", "user.email=t@t",
         "commit", "-q", "-m", "init driver")
    _git(repo, "-c", "user.name=t", "-c", "user.email=t@t",
         "commit", "-q", "--allow-empty", "-m", "placeholder",
         "--date", "2024-01-01")
    (repo / "driver.c").write_text(VULN_C_V2, encoding="utf-8")
    _git(repo, "add", "-A")
    _git(repo, "-c", "user.name=t", "-c", "user.email=t@t",
```

```
         "commit", "-q", "-m", "fix CVE-2024-1234: buffer overflow in sensor_set_name")
    fix1 = _git(repo, "rev-parse", "HEAD").strip()
    (repo / "net.c").write_text(
        "#include <stdint.h>\n"
        "int net_send(const void *d, int n) { (void)d; return n; }\n",
        encoding="utf-8")
    _git(repo, "add", "-A")
    _git(repo, "-c", "user.name=t", "-c", "user.email=t@t",
         "commit", "-q", "-m", "add net")
    (repo / "net.c").write_text(
        "#include <stdint.h>\n"
        "int net_send(const void *d, int n) { (void)d; return n * 2; }\n",
        encoding="utf-8")
    _git(repo, "add", "-A")
    _git(repo, "-c", "user.name=t", "-c", "user.email=t@t",
         "commit", "-q", "-m", "fix GHSA-xxxx-1: net_send length check")
    fix2 = _git(repo, "rev-parse", "HEAD").strip()
    return repo, fix1, fix2
def _manifest(repo: Path, fix1: str, fix2: str, path="manifest.json") -> Path:
    m = {
        "repo": str(repo),
        "license": "Apache-2.0",
        "advisories": [
            {"cve": "CVE-2024-1234", "cwe": "CWE-120", "fix_commit": fix1,
             "date": "2024-03-01", "severity_gb": "高危",
             "is_mmio": False, "is_rtos": False, "summary": "栈溢出"},
            {"cve": "CVE-2024-5678", "cwe": "CWE-190", "fix_commit": fix2,
             "date": "2024-06-01", "severity_gb": "中危",
             "is_rtos": True, "is_isr": False, "is_mmio": False,
             "summary": "长度整数溢出"},
        ],
    }
    p = repo / path
    p.write_text(json.dumps(m, ensure_ascii=False, indent=2), encoding="utf-8")
    return p
# ---------------- 单元：切片与指纹 ----------------
def test_extract_functions_slices():
    funcs = extract_functions(VULN_C_V1)
    names = [f["name"] for f in funcs]
    assert names == ["sensor_set_name", "sensor_ok"]
    body = funcs[0]["code"]
    assert "buf[i] = name[i];" in body and "#include" not in body
def test_fingerprint_ignores_comments_and_ws():
    a = fingerprint_text("int f(void) { return 1; }  /* 注释 */")
    b = fingerprint_text("int f(void)\n{\n    return 1;   // 另一注释\n}")
    assert a == b
def test_fingerprint_distinguishes_code():
    a = fingerprint_text("int f(void) { return 1; }")
    b = fingerprint_text("int f(void) { return 2; }")
    assert a != b
```

```
# ---------------- 单元：修复提交检索 ----------------
def test_find_fix_commits(demo_repo):
    repo, _, _ = demo_repo
    fixes = find_fix_commits(repo)
    subjects = [f["subject"] for f in fixes]
    assert any("CVE-2024-1234" in s for s in subjects)
    assert any("GHSA" in s for s in subjects)
# ---------------- GHSA 在线抓取（注入式 opener，无网络） ----------------
class _FakeOpener:
    """模拟 urllib.urlopen：返回预置 GHSA JSON。"""
    def __init__(self, payload):
        self.payload = payload
        self.request = None
    def __call__(self, req, timeout=0):
        self.request = req
        import io
        return io.BytesIO(json.dumps(self.payload).encode("utf-8"))
GHSA_PAYLOAD = [
    {
        "ghsa_id": "GHSA-xxxx-yyyy",
        "cve_id": "CVE-2024-1234",
        "severity": "HIGH",
        "summary": "buffer overflow in sensor_set_name",
        "published_at": "2024-03-01T00:00:00Z",
        "cwes": [{"cwe_id": "CWE-120", "name": "Classic Buffer Overflow"}],
    },
    {
        "ghsa_id": "GHSA-zzzz-0000",
        "cve_id": None,                      # 无 CVE 映射 → 应被跳过
        "severity": "LOW",
        "summary": "weak random",
        "published_at": "2024-05-01T00:00:00Z",
        "cwes": [],
    },
]
def test_fetch_ghsa_parses_and_maps():
    opener = _FakeOpener(GHSA_PAYLOAD)
    items = fetch_ghsa_advisories("acme/rtos", token="t0k", _opener=opener)
    assert "api.github.com/repos/acme/rtos" in opener.request.full_url
    assert opener.request.headers["Authorization"] == "Bearer t0k"
    assert len(items) == 2
    assert items[0]["cve"] == "CVE-2024-1234"
    assert items[0]["severity"] == "高危"          # GB/T 口径映射
    assert items[0]["cwes"] == ["CWE-120"]
    assert items[1]["cwes"] == ["CWE-OTHER"]       # 缺 CWE 回填
def test_ghsa_to_advisories_aligns_git(demo_repo):
    repo, fix1, _ = demo_repo
    items = [it for it in fetch_ghsa_advisories(
        "acme/rtos", _opener=_FakeOpener(GHSA_PAYLOAD))]
    advisories = ghsa_to_advisories(repo, items)
```

```
    assert len(advisories) == 1                    # 无 CVE 条目被跳过
    adv = advisories[0]
    assert adv.fix_commit == fix1                  # CVE 定位到修复提交
    assert adv.cwe == "CWE-120" and adv.severity_gb == "高危"
# ---------------- 集成：样本构建 ----------------
def test_build_samples_end_to_end(demo_repo, tmp_path):
    repo, fix1, fix2 = demo_repo
    mpath = _manifest(repo, fix1, fix2)
    manifest = load_manifest(mpath)
    samples, stats = build_samples(manifest, pair_ratio=5, seed=42)
    labels = {s.label for s in samples}
    assert {"vulnerable", "fixed", "normal"} <= labels
    vuln = [s for s in samples if s.label == "vulnerable"]
    assert any(s.func_name == "sensor_set_name" and s.cwe == "CWE-120"
               for s in vuln)
    # 漏洞函数与修复函数指纹不同（代码确实变化）
    v = next(s for s in vuln if s.func_name == "sensor_set_name")
    fixed = next(s for s in samples if s.label == "fixed"
                 and s.func_name == "sensor_set_name")
    assert v.fingerprint != fixed.fingerprint
    # 近重复删除：全部指纹唯一
    fps = [s.fingerprint for s in samples]
    assert len(fps) == len(set(fps))
    # 嵌入式上下文标注透传 + ISR 自动识别
    assert all(hasattr(s, "is_isr") for s in samples)
    assert stats["by_cwe"]["CWE-120"] >= 2
    # 时间划分：6:2:2 → 2 个通告分属两集合，无时间泄漏
    splits = {s.split for s in samples}
    assert splits <= {"train", "val", "test"}
    v_splits = {s.split for s in vuln}
    assert "train" in v_splits
def test_write_dataset_outputs(demo_repo, tmp_path):
    repo, fix1, fix2 = demo_repo
    manifest = load_manifest(_manifest(repo, fix1, fix2,
                                       path="manifest2.json"))
    samples, stats = build_samples(manifest, seed=42)
    out = tmp_path / "ds"
    write_dataset(samples, stats, manifest, out)
    lines = (out / "samples.jsonl").read_text(encoding="utf-8").splitlines()
    assert len(lines) == len(samples)
    rec = json.loads(lines[0])
    assert {"sample_id", "cwe", "label", "code", "fingerprint", "split",
            "severity_gb", "is_isr"} <= set(rec)
    assert (out / "stats.json").exists()
    assert (out / "LICENSES.md").exists()
    assert "Apache-2.0" in (out / "LICENSES.md").read_text(encoding="utf-8")
/* ==== tests/test_webui.py ==== */
# -*- coding: utf-8 -*-
"""Web 控制台测试：三源注册 / 后台扫描 / API 全流程。"""
import io
```

```
import json
import time
import urllib.request
import zipfile
from pathlib import Path
import pytest
from ecdet.webui import Store, WebUI
EXAMPLES = Path(__file__).parent.parent / "examples"
@pytest.fixture()
def ui(tmp_path):
    ui = WebUI(tmp_path / "data", port=0)
    httpd = ui.make_server()
    thread = threading_start(httpd)
    # 端口 0 → 实际端口取回
    port = httpd.server_address[1]
    yield ui, f"http://127.0.0.1:{port}"
    httpd.shutdown()
def threading_start(httpd):
    t = __import__("threading").Thread(target=httpd.serve_forever, daemon=True)
    t.start()
    return t
def _get(base, path):
    with urllib.request.urlopen(base + path, timeout=10) as r:
        return json.loads(r.read().decode("utf-8"))
def _post(base, path, payload=None, raw=None, ctype="application/json"):
    if raw is not None:
        body = raw
    else:
        body = json.dumps(payload or {}, ensure_ascii=False).encode("utf-8")
    req = urllib.request.Request(base + path, data=body, method="POST",
                                 headers={"Content-Type": ctype})
    with urllib.request.urlopen(req, timeout=30) as r:
        return json.loads(r.read().decode("utf-8"))
def _wait_run(base, run_id, timeout=120):
    deadline = time.time() + timeout
    while time.time() < deadline:
        run = _get(base, f"/api/runs/{run_id}")
        if run["status"] in ("done", "failed"):
            return run
        time.sleep(0.5)
    raise TimeoutError("scan run timeout")
# ---------------- Store 单元 ----------------
def test_store_roundtrip(tmp_path):
    store = Store(tmp_path / "db.sqlite")
    pid = store.add_project("p1", "path", "ref", "ws")
    run_id = store.add_run(pid, {"triage": "off"})
    store.set_run_status(run_id, "running")
    store.put_alerts(run_id, [
        {"rule_id": "R1", "engine": "builtin", "severity": "error",
         "path": "a.c", "line": 3, "message": "m1", "cwe": "CWE-1",
```

```
         "provenance": "detected", "fingerprint": "fp1"},
        {"rule_id": "R2", "engine": "dynamic", "severity": "warning",
         "path": "b.c", "line": 9, "message": "m2", "cwe": None,
         "provenance": "suspected", "fingerprint": "fp2",
         "triage": {"verdict": "TRUE_POSITIVE"}},
    ])
    alerts = store.list_alerts(run_id)
    assert len(alerts) == 2
    assert alerts[0]["rule_id"] == "R1"          # error 排前
    assert store.list_alerts(run_id, severity="warning")[0]["rule_id"] == "R2"
    assert store.list_alerts(run_id, q="m1")[0]["rule_id"] == "R1"
    assert store.set_alert_status(alerts[0]["id"], "confirmed")
    assert store.list_alerts(run_id, status="confirmed")[0]["rule_id"] == "R1"
    assert not store.set_alert_status(alerts[0]["id"], "bogus")
    counts = store.alert_counts(run_id)
    assert counts["by_severity"]["error"] == 1
    assert counts["by_status"]["confirmed"] == 1
def test_store_local_overrides_global(tmp_path):
    store = Store(tmp_path / "db.sqlite")
    store.put_alerts(1, [
        {"rule_id": "R", "engine": "e", "severity": "note", "path": "a.c",
         "line": 1, "message": "x", "cwe": None, "provenance": "detected",
         "fingerprint": ""}])
    assert store.list_alerts(1)[0]["status"] == "pending"
# ---------------- 注册归一 ----------------
def test_register_path_source(ui):
    ui_obj, base = ui
    resp = _post(base, "/api/projects",
                 {"name": "demo", "source_type": "path",
                  "path": str(EXAMPLES / "demo_rtos")})
    assert resp["project_id"] >= 1
    projects = _get(base, "/api/projects")["projects"]
    assert any(p["name"] == "demo" for p in projects)
def test_register_path_missing(ui):
    _, base = ui
    req = urllib.request.Request(
        base + "/api/projects",
        data=json.dumps({"name": "x", "source_type": "path",
                         "path": "Z:/no/such/dir"}).encode(),
        method="POST", headers={"Content-Type": "application/json"})
    try:
        urllib.request.urlopen(req, timeout=10)
        raise AssertionError("应返回 400")
    except urllib.error.HTTPError as e:
        assert e.code == 400
def test_register_upload_zip(ui):
    ui_obj, base = ui
    buf = io.BytesIO()
    with zipfile.ZipFile(buf, "w") as zf:
        zf.writestr("pkg/src/main.c", "int main(void) { return 0; }\n")
```

```
        zf.writestr("pkg/src/../evil.c", "bad")   # 路径逃逸应被过滤
    resp = _post(base, "/api/projects?name=zipped", raw=buf.getvalue(),
                 ctype="application/zip")
    ws = Path(resp["workspace"])
    assert (ws / "pkg" / "src" / "main.c").exists()
    assert not (ws / "evil.c").exists() and not (ws / "pkg" / "evil.c").exists()
def test_register_upload_no_sources(ui):
    _, base = ui
    buf = io.BytesIO()
    with zipfile.ZipFile(buf, "w") as zf:
        zf.writestr("readme.txt", "no code here")
    req = urllib.request.Request(
        base + "/api/projects?name=empty", data=buf.getvalue(), method="POST",
        headers={"Content-Type": "application/zip"})
    try:
        urllib.request.urlopen(req, timeout=10)
        raise AssertionError("应返回 400")
    except urllib.error.HTTPError as e:
        assert e.code == 400
# ---------------- 扫描全流程（API） ----------------
def test_full_scan_flow(ui):
    ui_obj, base = ui
    reg = _post(base, "/api/projects",
                {"name": "demo", "source_type": "path",
                 "path": str(EXAMPLES / "demo_rtos")})
    run = _post(base, "/api/runs", {"project_id": reg["project_id"],
                                    "triage": "off"})
    done = _wait_run(base, run["run_id"])
    assert done["status"] == "done", done.get("error")
    assert done["stats"]["total"] > 0
    assert done["alert_counts"]["by_severity"].get("error", 0) >= 1
    alerts = _get(base, f"/api/runs/{run['run_id']}/alerts")["alerts"]
    assert len(alerts) == done["stats"]["total"]
    assert any(a["rule_id"] == "CERT C STR31-C" for a in alerts)
    # 筛选
    only_err = _get(base,
                    f"/api/runs/{run['run_id']}/alerts?severity=error")["alerts"]
    assert all(a["severity"] == "error" for a in only_err)
    # 状态流转
    aid = alerts[0]["id"]
    _post(base, f"/api/alerts/{aid}/status", {"status": "confirmed"})
    got = _get(base, f"/api/runs/{run['run_id']}/alerts?status=confirmed")["alerts"]
    assert any(a["id"] == aid for a in got)
    # 报告产物可访问
    with urllib.request.urlopen(
            base + f"/api/runs/{run['run_id']}/report", timeout=10) as r:
        html = r.read().decode("utf-8")
        assert "嵌入式代码检测报告" in html
    with urllib.request.urlopen(
            base + f"/api/runs/{run['run_id']}/sarif", timeout=10) as r:
```

```
        sarif = json.loads(r.read().decode("utf-8"))
        assert sarif["version"] == "2.1.0"
    # 运行列表
    runs = _get(base, f"/api/runs?project_id={reg['project_id']}")["runs"]
    assert runs and runs[0]["id"] == run["run_id"]
def test_full_scan_with_mock_triage(ui):
    ui_obj, base = ui
    reg = _post(base, "/api/projects",
                {"name": "demo2", "source_type": "path",
                 "path": str(EXAMPLES / "demo_rtos")})
    run = _post(base, "/api/runs", {"project_id": reg["project_id"],
                                    "triage": "mock"})
    done = _wait_run(base, run["run_id"])
    assert done["status"] == "done"
    alerts = _get(base, f"/api/runs/{run['run_id']}/alerts?severity=error")["alerts"]
    triaged = [a for a in alerts if a["triage"]]
    assert triaged, "Mock 分诊后 error 级告警应携带 triage 结论"
def test_carriers_endpoint(ui):
    _, base = ui
    data = _get(base, "/api/carriers")
    names = {c["name"] for c in data["carriers"]}
    assert names == {"host", "unicorn", "renode"}
def test_index_page(ui):
    _, base = ui
    with urllib.request.urlopen(base + "/", timeout=10) as r:
        html = r.read().decode("utf-8")
        assert "ecdet 控制台" in html
# ---------------- V0.9：总览聚合 / 告警详情（证据链） / 静态资源 ----------------
def test_summary_and_alert_detail(ui):
    ui_obj, base = ui
    reg = _post(base, "/api/projects",
                {"name": "demo", "source_type": "path",
                 "path": str(EXAMPLES / "demo_rtos")})
    run = _post(base, "/api/runs", {"project_id": reg["project_id"],
                                    "triage": "mock"})
    done = _wait_run(base, run["run_id"])
    assert done["status"] == "done"
    s = _get(base, "/api/summary")
    assert s["projects"] >= 1 and s["runs_done"] >= 1
    assert s["alerts_total"] == done["stats"]["total"]
    assert set(s["by_severity"]) <= {"error", "warning", "note"}
    assert len(s["recent_runs"]) >= 1
    alerts = _get(base, f"/api/runs/{run['run_id']}/alerts?severity=error")["alerts"]
    detail = _get(base, f"/api/alerts/{alerts[0]['id']}")
    assert detail["rule_id"] == alerts[0]["rule_id"]
    assert isinstance(detail["evidence"], list)
    assert any("证据" in e["message"] or e["message"] for e in detail["evidence"])
def test_static_assets_served(ui):
    _, base = ui
    for path, marker in (("/static/style.css", "--sidebar"),
```

```
                        ("/static/app.js", "ecdet")):
        with urllib.request.urlopen(base + path, timeout=10) as r:
            body = r.read().decode("utf-8")
            assert marker in body
    try:
        urllib.request.urlopen(base + "/static/../secrets", timeout=10)
    except urllib.error.HTTPError as e:
        assert e.code in (400, 404)
# ---------------- V1.0：设置 / 一键修复 ----------------
def test_settings_roundtrip_mask(ui):
    _, base = ui
    _post(base, "/api/settings", {"llm_mode": "real",
                                  "llm_base_url": "http://127.0.0.1:9/v1",
                                  "llm_api_key": "sk-test-1234",
                                  "llm_model": "test-model"})
    s = _get(base, "/api/settings")
    assert s["llm_mode"] == "real"
    assert s["llm_base_url"] == "http://127.0.0.1:9/v1"
    assert s["llm_model"] == "test-model"
    assert s["api_key_set"] is True
    assert "1234" in s["api_key_masked"] and "sk-test" not in s["api_key_masked"]
    # 空 key 不覆盖旧值
    _post(base, "/api/settings", {"llm_model": "model-2"})
    s2 = _get(base, "/api/settings")
    assert s2["llm_model"] == "model-2" and s2["api_key_set"] is True
def test_settings_test_endpoint(ui):
    """测试连接：起一个 OpenAI 格式的假端点验证探针回路。"""
    ui_obj, base = ui
    import threading
    from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
    class FakeOpenAI(BaseHTTPRequestHandler):
        def log_message(self, *a):
            pass
        def do_POST(self):
            body = b'{"choices":[{"message":{"content":"pong"}}]}'
            self.send_response(200)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)
    httpd = ThreadingHTTPServer(("127.0.0.1", 0), FakeOpenAI)
    threading.Thread(target=httpd.serve_forever, daemon=True).start()
    port = httpd.server_address[1]
    try:
        _post(base, "/api/settings", {"llm_mode": "real",
                                      "llm_base_url": f"http://127.0.0.1:{port}/v1",
                                      "llm_api_key": "sk-x",
                                      "llm_model": "fake"})
        r = _post(base, "/api/settings/test")
        assert r["ok"] is True and "pong" in r["message"]
```

```
    finally:
        httpd.shutdown()
def test_one_click_fix_flow(ui):
    """扫描（Mock 分诊）→ 一键修复 → 采纳补丁含 diff（STR31→strncpy）。"""
    _, base = ui
    reg = _post(base, "/api/projects",
                {"name": "fixdemo", "source_type": "path",
                 "path": str(EXAMPLES / "demo_rtos")})
    run = _post(base, "/api/runs", {"project_id": reg["project_id"],
                                    "triage": "mock"})
    done = _wait_run(base, run["run_id"])
    assert done["status"] == "done"
    job = _post(base, f"/api/runs/{run['run_id']}/fix")
    assert job["job_id"] >= 1
    deadline = time.time() + 120
    while time.time() < deadline:
        data = _get(base, f"/api/runs/{run['run_id']}/fixes")
        j = data["job"]
        if j and j["status"] in ("done", "failed"):
            break
        time.sleep(0.5)
    assert j["status"] == "done", j.get("error")
    assert j["accepted"] >= 1
    str31 = [p for p in data["proposals"]
             if p["rule_id"] == "CERT C STR31-C" and p["verified"]]
    assert str31, "STR31 正确补丁应被验证采纳"
    assert "strncpy" in str31[0]["diff"]
    # 告警详情接口携带 run_id（抽屉按 alert_id 匹配补丁）
    alerts = _get(base, f"/api/runs/{run['run_id']}/alerts")["alerts"]
    target = next(a for a in alerts if a["rule_id"] == "CERT C STR31-C")
    detail = _get(base, f"/api/alerts/{target['id']}")
    assert detail["run_id"] == run["run_id"]
def test_targeted_fix_by_alert_ids(ui):
    """勾选定向修复：只修选中的告警，且原文件内容保持不变。"""
    _, base = ui
    reg = _post(base, "/api/projects",
                {"name": "tf", "source_type": "path",
                 "path": str(EXAMPLES / "demo_rtos")})
    run = _post(base, "/api/runs", {"project_id": reg["project_id"],
                                    "triage": "mock"})
    done = _wait_run(base, run["run_id"])
    assert done["status"] == "done"
    alerts = _get(base, f"/api/runs/{run['run_id']}/alerts")["alerts"]
    str31 = next(a for a in alerts if a["rule_id"] == "CERT C STR31-C")
    target_file = EXAMPLES / "demo_rtos" / "src" / "sensor.c"
    before = target_file.read_bytes()
    # 只勾 STR31 与 ARR30 两条
    arr30 = next(a for a in alerts if a["rule_id"] == "CERT C ARR30-C")
    job = _post(base, f"/api/runs/{run['run_id']}/fix",
                {"alert_ids": [str31["id"], arr30["id"]]})
```

```
    deadline = time.time() + 120
    data = {}
    while time.time() < deadline:
        data = _get(base, f"/api/runs/{run['run_id']}/fixes")
        if data["job"] and data["job"]["status"] in ("done", "failed"):
            break
        time.sleep(0.5)
    assert data["job"]["status"] == "done", data["job"].get("error")
    assert len(data["proposals"]) == 2, "只应生成勾选的 2 条建议"
    assert {p["rule_id"] for p in data["proposals"]} ==         {"CERT C STR31-C", "CERT C ARR30-C"}
    # v1.2：补丁记录回填 alert_id，告警抽屉按 id 匹配展示
    str31_fix = next(p for p in data["proposals"]
                     if p["rule_id"] == "CERT C STR31-C")
    assert str31_fix["alert_id"] == str31["id"]
    # 原文件一个字节都不能变（修复只生成补丁不落盘）
    assert target_file.read_bytes() == before
def test_apply_mode_writes_and_converges(ui):
    """自动修复模式：验证通过的补丁写入源文件，验收扫描告警下降，
    原始 examples 不受影响（工程以 zip 副本注册）。"""
    ui_obj, base = ui
    import io
    buf = io.BytesIO()
    src_root = EXAMPLES / "demo_rtos"
    with zipfile.ZipFile(buf, "w") as zf:
        for f in src_root.rglob("*"):
            if f.is_file() and f.suffix in (".c", ".h", ".txt"):
                zf.writestr(f.relative_to(src_root).as_posix(),
                            f.read_text(encoding="utf-8"))
    reg = _post(base, "/api/projects?name=applydemo",
                raw=buf.getvalue(), ctype="application/zip")
    ws = Path(reg["workspace"])
    run = _post(base, "/api/runs", {"project_id": reg["project_id"],
                                    "triage": "off"})
    _wait_run(base, run["run_id"])
    before_err = None
    job = _post(base, f"/api/runs/{run['run_id']}/fix", {"mode": "apply"})
    data = {}
    deadline = time.time() + 180
    while time.time() < deadline:
        data = _get(base, f"/api/runs/{run['run_id']}/fixes")
        if data["job"] and data["job"]["status"] in ("done", "failed"):
            break
        time.sleep(0.5)
    j = data["job"]
    assert j["status"] == "done", j.get("error")
    assert j["mode"] == "apply"
    assert j["applied"] >= 2, "STR31/CWE-197/CWE-1284 至少两条应写入"
    assert j["rounds"] >= 1
    fs = j["final_stats"]
    assert fs["remaining_error"] < 6, "自动修复后 error 应下降"
```

```
    assert fs["applied_total"] == j["applied"]
    # 源文件已被真实修改
    sensor = (ws / "src" / "sensor.c").read_text(encoding="utf-8")
    assert "strncpy" in sensor, "STR31 补丁应已写入"
    assert "uint16_t gain" in sensor, "CWE-197 类型拓宽应已写入"
    comm = (ws / "src" / "comm.c").read_text(encoding="utf-8")
    assert "%15s" in comm, "CWE-1284 限宽补丁应已写入"
    # 备份存在且内容为修复前版本
    backup = Path(j["backup_dir"])
    assert backup.exists()
    bak_sensor = (backup / "src" / "sensor.c").read_text(encoding="utf-8")
    assert "strncpy" not in bak_sensor and "strcpy" in bak_sensor
    # 原始 examples 目录不受影响
    orig = (src_root / "src" / "sensor.c").read_text(encoding="utf-8")
    assert "strncpy" not in orig
# ---------------- V1.2：源码快照 / 代码浏览 / 版本对比 ----------------
def _zip_project(src_root: Path) -> bytes:
    buf = io.BytesIO()
    with zipfile.ZipFile(buf, "w") as zf:
        for f in src_root.rglob("*"):
            if f.is_file() and f.suffix in (".c", ".h", ".txt"):
                zf.writestr(f.relative_to(src_root).as_posix(),
                            f.read_text(encoding="utf-8"))
    return buf.getvalue()
def test_snapshot_skip_guard(tmp_path):
    """快照护栏：只收 C/H、超限单文件跳过、manifest 记录 sha256。"""
    from ecdet.webui.snapshot import (MAX_FILE_BYTES, create_snapshot,
                                      load_manifest)
    proj = tmp_path / "proj"
    (proj / "src").mkdir(parents=True)
    (proj / "src" / "ok.c").write_text("int x;\n", encoding="utf-8")
    (proj / "src" / "big.c").write_text("x" * (MAX_FILE_BYTES + 1))
    (proj / "src" / "note.txt").write_text("not source", encoding="utf-8")
    s = create_snapshot(tmp_path, 1, proj)
    assert s["files"] == 1 and s["skipped"] == 1
    m = load_manifest(tmp_path, 1)
    assert "src/ok.c" in m["files"]
    assert "src/big.c" not in m["files"]
    assert m["files"]["src/ok.c"]["lines"] == 1
def test_snapshot_files_and_file_api(ui):
    """扫描后快照可浏览：files 含告警计数；file 返回整文件与告警行；
    非 manifest 路径 404（防穿越）。"""
    _, base = ui
    reg = _post(base, "/api/projects?name=snapdemo",
                raw=_zip_project(EXAMPLES / "demo_rtos"),
                ctype="application/zip")
    run = _post(base, "/api/runs", {"project_id": reg["project_id"]})
    done = _wait_run(base, run["run_id"])
    assert done["status"] == "done"
    assert done["stats"].get("snapshot", {}).get("files", 0) >= 1
```

```
    data = _get(base, f"/api/runs/{run['run_id']}/files")
    assert data["snapshot"]["files"] >= 1
    paths = {f["path"] for f in data["files"]}
    assert "src/sensor.c" in paths
    sensor = next(f for f in data["files"] if f["path"] == "src/sensor.c")
    assert sensor["alerts"] >= 1
    body = _get(base,
                f"/api/runs/{run['run_id']}/file?path=src%2Fsensor.c")
    assert len(body["lines"]) > 10
    assert body["alerts"], "sensor.c 应携带告警标注"
    assert all(a["line"] >= 1 for a in body["alerts"])
    for bad in ("../../ecdet_web.db", "nope/ghost.c"):
        try:
            _get(base, f"/api/runs/{run['run_id']}/file?path={bad}")
            raise AssertionError("非法 path 应返回 404")
        except urllib.error.HTTPError as e:
            assert e.code == 404
def test_diff_api_between_runs(ui):
    """两 run 对比：新增文件 + 新增告警（STR31）→ 文件级与行级 diff。"""
    _, base = ui
    reg = _post(base, "/api/projects?name=diffdemo",
                raw=_zip_project(EXAMPLES / "demo_rtos"),
                ctype="application/zip")
    ws = Path(reg["workspace"])
    run1 = _post(base, "/api/runs", {"project_id": reg["project_id"]})
    _wait_run(base, run1["run_id"])
    (ws / "extra.c").write_text(
        "#include <string.h>\n#include <stdio.h>\n"
        "void extra_copy(const char *src) {\n    char buf[8];\n"
        "    (void)src;\n"
        "    strcpy(buf, \"0123456789abcdef0123456789abcdef\");\n"
        "    printf(\"%s\\n\", buf);\n}\n",
        encoding="utf-8")
    run2 = _post(base, "/api/runs", {"project_id": reg["project_id"]})
    _wait_run(base, run2["run_id"])
    d = _get(base,
             f"/api/runs/{run2['run_id']}/diff?base={run1['run_id']}")
    s = d["summary"]
    assert s["added"] == 1 and s["modified"] == 0
    assert s["alerts_new"] >= 1
    assert any(f["path"] == "extra.c" and f["status"] == "added"
               for f in d["files"])
    assert "CERT C STR31-C" in {a["rule_id"] for a in d["alerts_new"]}
    fd = _get(base,
              f"/api/runs/{run2['run_id']}/diff/file"
              f"?base={run1['run_id']}&path=extra.c")
    assert fd["status"] == "added"
    assert fd["rows"] and all(r["t"] == "add" for r in fd["rows"])
    assert any(a["rule_id"] == "CERT C STR31-C" for a in fd["new_alerts"])
    # 基准与目标相同 → 400
```

```
try:
    _get(base, f"/api/runs/{run2['run_id']}/diff?base={run2['run_id']}")
    raise AssertionError("同 run 对比应返回 400")
except urllib.error.HTTPError as e:
    assert e.code == 400
```